

Information Visualization

Comprehensive Exam Study Guide

TU Wien, 2026S — based on the lecture slides and mandatory readings

*This guide is organised by content domain (Sections 1–12) followed by an exam playbook, an algorithm cheat-sheet, list-style quick references and practice questions (Sections 13–16). Boxes mark **key points**, **algorithm fact-sheets**, **exam angles** and **common point deductions**.*

Contents

1	Foundations of Information Visualization	5
1.1	What is Information Visualization?	5
1.1.1	What – Why – How	5
1.2	High-Level Goals: Exploration, Confirmation, Presentation	5
1.3	The InfoVis Reference Model (Pipeline)	6
1.4	Munzner’s What–Why–How Analysis Framework	6
1.5	Human-in-the-Loop vs Pure Automation	7
1.6	Resource Limitations	7
1.7	Visualization Disciplines and the Design Triangle	7
2	Data Abstraction	8
2.1	Semantics vs Type	8
2.2	The Five Data Types	8
2.3	The Four Dataset Types (+ Clusters/Sets/Lists)	9
2.4	Grid Types (for fields)	9
2.5	Attribute Types	9
2.5.1	Ordering Direction (for ordered attributes)	10
2.6	Keys vs Values; Flat vs Multidimensional Tables	10
2.7	Temporal and Time-Varying Semantics	10
2.8	Dataset Availability: Static vs Dynamic/Streaming	10
2.9	Raw Data Issues in Data Tables (exam “List” topic)	11
3	Visual Encoding: Marks and Channels	11
3.1	Marks	11
3.2	Channels and the magnitude vs. identity split	12
3.3	The two guiding principles	12
3.4	Channel effectiveness rankings	13
3.5	Accuracy: Cleveland & McGill and Stevens	13
3.6	Channel characteristics	14
3.7	Channel interference & redundant encoding	15
4	Perception and Color	15
4.1	The Human Visual System	15
4.1.1	Eye, retina, rods and cones	15
4.1.2	Fovea and acuity	16
4.1.3	Fixations and saccades	16
4.1.4	Visual cortex (it’s complicated)	16
4.2	Preattentive Processing and Popout	16

4.3	Gestalt Laws of Grouping	17
4.4	Weber's Law and the Just Noticeable Difference	17
4.4.1	JND	17
4.4.2	Weber's law	17
4.5	Color	18
4.5.1	Color is three channels	18
4.5.2	Opponent color theory	18
4.5.3	Colormap types and when to use each	18
4.5.4	The rainbow / jet colormap problem	19
4.5.5	Perceptually uniform maps (viridis)	19
5	Design Principles and Visualization Critique	20
5.1	Data-Ink Ratio and Chartjunk (Tufte)	20
5.2	The Lie Factor (graphical integrity)	20
5.3	Over-Counting via Area (1D value as 2D size)	21
5.4	Ordering of Bars, Axes and Categories	21
5.5	Expressiveness Violations	21
5.6	Effectiveness, Stevens' Power Law and the Channel Ranking	21
5.7	Channel Interference (separable vs. integral)	22
5.8	Eyes Beat Memory: Juxtaposition vs. Animation	22
5.9	Problems with 3D	23
5.10	Function Before Form	23
5.11	The Critique Checklist (use on any chart)	23
5.12	Worked Critique 1: Rainbow Choropleth (Texas map)	23
5.13	Worked Critique 2: Area-Encoded Circles	24
6	Tasks and Interaction	24
6.1	Task Abstraction: Actions (Munzner ch. 3)	25
6.1.1	High level – Analyze (consume vs. produce)	25
6.1.2	Mid level – Search (a 2×2 of known/unknown)	25
6.1.3	Low level – Query (scope: one / some / all)	25
6.2	Task Abstraction: Targets	26
6.3	Interaction Techniques	26
6.3.1	Shneiderman's Visual Information-Seeking Mantra	26
6.3.2	Zoom: geometric vs. semantic; pan	26
6.3.3	Overview+Detail vs. Focus+Context	26
6.3.4	Response-time budget	27
6.3.5	Seven categories of interaction (Yi et al. 2007)	27
6.4	Linked Views / Coordinated Multiple Views (CMV)	27
6.5	High-Dimensional Brushing (Ward, XmdvTool)	28
6.6	Quantitative Visual Analytics (Radoš et al. 2016)	28
7	High-Dimensional Data Visualization	29
7.1	Curse of Dimensionality (Giraud)	30
7.2	Feature Selection vs. Feature Extraction (Ghojogh)	30
7.3	Visual Methods for Multivariate Tabular Data	31
7.4	Ranking Views: Scagnostics & Class Consistency	32
7.5	Dimensionality Reduction Fact-Sheets	33
7.5.1	Comparison Table: PCA vs MDS vs t-SNE vs UMAP	35

8	Scalable Information Visualization & Big Data	35
8.1	Factors Affecting (Visual) Scalability	36
8.2	Overplotting: Simple Solutions vs. Preprocessing	36
8.3	Aggregation & Density: Scalable Encodings	37
8.4	Data Cubes / Multi-Dimensional Tables (OLAP)	38
8.5	Reducing Processing & Rendering Latency	38
8.6	Reduce Strategy (Munzner ch. 13)	39
8.7	Interactive Exploration: Shneiderman’s Mantra	39
9	Network and Tree Visualization	40
9.1	Graph basics	40
9.2	Node-link vs. adjacency matrix (mapping & alternatives)	40
9.3	Aesthetic criteria for graph drawing (the “List” topic)	41
9.4	Layout algorithm: Force-directed / spring embedder	41
9.5	Layout algorithm: Layered / Sugiyama (hierarchies, flow)	42
9.6	Dynamic graphs and the mental map	42
9.7	Tree layouts	43
9.8	Biological networks (Kerren)	43
9.9	Exam boxes	44
10	Complex Data: Sets, Curves, and Ensembles	44
10.1	Set-Typed Data	45
10.1.1	Techniques (Sets-STAR categories)	45
10.1.2	Comparison of set-visualisation categories	47
10.2	Families of Curves / Function Graphs	47
10.3	Simulation Ensembles	48
10.4	Exam Angle	49
11	Visualization for Machine Learning (VIS4ML) & Explainable AI	49
11.1	Why VIS4ML: visualization in the ML pipeline	50
11.2	Confusion matrix – “IF the model learned correctly”	50
11.3	Feature vectors from neuron activations (Rauber et al.)	51
11.4	Interpretability taxonomy (XAI)	51
11.5	Model-agnostic methods	52
11.6	Surrogate models	53
12	Evaluation and Validation	53
12.1	Munzner’s Nested Model: Four Levels	54
12.1.1	Threats and Validation per Level	54
12.2	Types of Evaluation	55
12.3	Controlled Experiment Design	56
12.3.1	Experimental Design and Statistics	56
12.4	Worked Example: Encoding Classes in Static Scatterplots	57
12.5	Effectiveness: Why We Visualize Defines the DV	57
13	Exam Question-Type Playbook	58
13.1	Type 1 – Visualization Literacy (read a chart)	58
13.2	Type 2 – Visualization Algorithms (describe / sketch / MCQ)	58
13.3	Type 3 – Visualization Strategy (data + task → design)	59
13.4	Type 4 – Design & Validation Strategy (user study)	59
13.5	Type 5 – Critique & Suggestions	60
13.6	Type 6 – Alternatives (compare two views)	60
13.7	Type 7 – Transfer / Sketch	61

13.8 Type 8 – Mapping Data Items Between Views	61
13.9 Type 9 – List / Enumerate	62
14 Algorithm Cheat-Sheet	62
15 List-Question Quick Reference	65
15.1 Raw-data issues in data tables	65
15.2 Factors affecting scalability	65
15.3 Sources of latency	65
15.4 Aesthetic criteria for graph drawing	66
15.5 Overplotting fixes that need no preprocessing	66
15.6 Model-agnostic AI interpretability methods	66
15.7 Techniques to visualize set-typed data	66
15.8 Methods for validation of visual encoding and/or interaction design	67
15.9 Typical dependent variables in user studies	67
16 Practice Questions with Model Answers	67

1 Foundations of Information Visualization

1.1 What is Information Visualization?

The standard definition used in the lecture is from Card, Mackinlay & Shneiderman:

Definition

Information Visualization is “the use of *computer-supported, interactive, visual* representations of *abstract data* to *amplify cognition*.” [Card et al., 1999]

Each highlighted word is load-bearing:

- **computer-supported** – the visualization is generated and (re-)rendered by a computer, not drawn once by hand.
- **interactive** – the user can manipulate the view (filter, zoom, select, re-map). Interactivity is what distinguishes InfoVis from a static print figure.
- **visual representation** – data is encoded into visual marks and channels so the human visual system can read it.
- **abstract data** – the data has *no inherent spatial mapping* (tables, networks, text), in contrast to scientific visualization where space is given (medical scans, flow fields).
- **amplify cognition** – the purpose. *Cognition* = “the mental process of acquiring knowledge and understanding through thought, experience, and the senses” [Oxford]. A later Card quote sharpens the point: “The purpose of information visualization is to amplify cognitive performance, *not just to create interesting pictures*.”

1.1.1 What – Why – How

The lecture frames the whole field with three questions (this maps directly onto Munzner’s framework, see below):

- **What?** Abstract data.
- **Why?** To amplify cognition.
- **How?** Visual representations + interactivity.

Mechanistically, visualization **expands human working memory**: instead of holding facts in the head, the user offloads them onto an external display (**external cognition**). The design challenge is to find the **most expressive and effective** visualization, turning insights into knowledge.

1.2 High-Level Goals: Exploration, Confirmation, Presentation

Visualization serves three high-level goals, ordered here by how much **interactivity** they typically need (most → least):

Goal	Description	Interactivity
Exploration	Searching / analysing data <i>without</i> a prior hypothesis, to find potentially useful info	high
Confirmation	Goal-oriented examination of <i>hypotheses</i>	medium
Presentation	Efficiently & effectively <i>communicate</i> facts	low

Sketch: Draw a horizontal triangle/wedge for “interactivity” that is thick on the left (Exploration) and thin on the right (Presentation); label the three goals underneath.

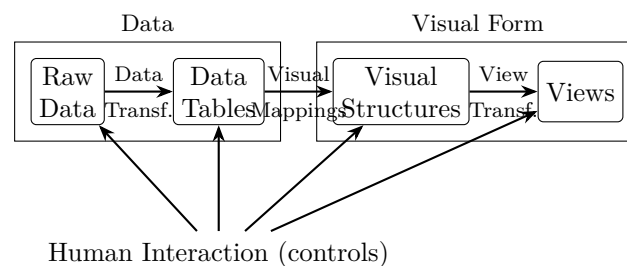
Exploration is the open-ended, *exploratory* mode; confirmation and presentation are the *explanatory / confirmatory* modes. A dashboard (e.g. Tableau) is exploratory; John Snow’s 1854 cholera map is confirmatory/presentational; a New York Times infographic is presentational.

Why we still need visualization – Anscombe / Datasaurus

Anscombe’s Quartet (and the **Datasaurus Dozen**) are four/twelve datasets with *identical summary statistics* – same mean ($\bar{x}=9, \bar{y}=7.5$), same variance ($s_x^2=11, s_y^2=4.12$), same correlation, same linear regression ($y = 0.5x + 3$) – yet *completely different shapes* when plotted. Summary statistics (and an LLM that only reads the numbers) can be badly misled; only the visualization reveals curvature, outliers, and clustering.

1.3 The InfoVis Reference Model (Pipeline)

The lecture’s “How to visualize?” answer is the classic **InfoVis reference model** (Card / Mackinlay): a pipeline of four data states connected by three transforms, all steerable by the user.



- **Raw Data** $\xrightarrow{\text{Data Transformations}}$ **Data Tables** – clean, derive, restructure (the “Data” stage; covered in Section 2).
- **Data Tables** $\xrightarrow{\text{Visual Mappings}}$ **Visual Structures** – assign spatial substrate, marks and visual channels (the key step; “Visual Form”).
- **Visual Structures** $\xrightarrow{\text{View Transformations}}$ **Views** – navigation, zoom, distortion, selection.
- **Human Interaction (controls)** feeds back into *every* transform, and the whole loop is driven by the user’s **Task**.

Common pitfall

Do not collapse the pipeline. The three transforms are distinct: *data transformation* ($\neq$ visual mapping) happens *before* anything visual exists; *visual mapping* is data $\rightarrow$ marks/channels; *view transformation* is on the already-rendered structure. Forgetting that interaction feeds back into all stages also loses points.

1.4 Munzner’s What–Why–How Analysis Framework

The mandatory reading (Munzner, 2014) structures any visualization design/analysis around three **nested** questions:

The three nested questions

- **What?** – the **data abstraction**: which dataset and attribute types are involved (Section 2).
- **Why?** – the **task abstraction**: what the user wants to accomplish (e.g. discover, present, compare, locate trends/outliers).
- **How?** – the **idiom**: **visual encoding** (marks + channels) and **interaction design** that solve the what/why.

Nested means the answers are dependent: the *how* you choose must fit the *what* and the *why*. A core skill (heavily tested) is to translate a *domain-specific* problem into this *abstract* vocabulary so that solutions transfer across domains. Example: “age, BMI, glucose, eGFR for some patients”

and “vehicles, accidents, deaths, population per Austrian state” are, abstractly, the *same* thing – a table of items with one categorical key and several quantitative attributes.

1.5 Human-in-the-Loop vs Pure Automation

When to visualize, when to automate

- **Pure automatic methods** work *only for clearly specified problems* (a well-defined question, a known model).
- **Pure visualization** is *not enough* for very large and complex data.
- Use **visualization / a human in the loop** when the problem is *ill-specified*, exploratory, or requires judgement, and when you need to leverage the human visual system to spot the unexpected.

This is the rationale for **Visual Analytics**: a feedback loop coupling automated data analysis (data mining, model building) with interactive visualization, so that **visual data exploration** (Data → Visualization → Knowledge, with user interaction and parameter refinement) and automated analysis reinforce each other. Classic example: an **outlier** that a regression line would average over is spotted visually, inspected with external context, and removed by the analyst.

How this is asked

Typical exam tasks for this section: (1) state and explain the Card definition word by word; (2) name and order the three high-level goals and link them to interactivity; (3) draw / label the InfoVis reference model and name the three transforms; (4) give Munzner’s three nested questions and explain why they are nested; (5) argue when visualization beats pure automation, often via Anscombe’s Quartet. Expect “define X” and “draw the pipeline” phrasing.

1.6 Resource Limitations

A good design respects three limited resources (Munzner). Trade-offs between them drive most design decisions.

Resource	Limit / consequence
Computer	computation time and memory; large data → slow or infeasible idioms
Human	attention, working memory, perceptual/cognitive capacity
Display	finite pixels (screen real estate); the information-density vs clutter trade-off

1.7 Visualization Disciplines and the Design Triangle

- **Information Visualization (InfoVis)** – spatial layout is *chosen* by the designer (abstract data).
- **Scientific Visualization (SciVis)** – spatial position is *given* with the data (continuous fields, e.g. medical scans).
- **Visual Analytics / Visual Data Science** – visualization + automated analysis in a human-in-the-loop.

The **design triangle** [Miksch & Aigner] ties it together: good interactive (visual analytics) methods sit between **data**, **users** and **tasks**, and must be **expressive** (w.r.t. data), **effective** (w.r.t. users/perception) and **appropriate** (w.r.t. tasks).

Minimalistic design – chartjunk

Tufte: “interior decoration of graphics generates a lot of ink that does not tell the viewer

anything new ... it is often **chartjunk**.” Maximise the **data-ink ratio** = $\frac{\text{data-ink}}{\text{total ink}}$. (Caveat from Bateman et al. CHI 2010: embellished “infographic” charts had comparable interpretation accuracy and *better long-term recall* – so chartjunk is not always strictly worse, but for analysis tasks favour minimal designs.)

2 Data Abstraction

This section answers Munzner’s **What?** question: the abstract description of the data, independent of any domain. The take-away from the lecture is to **describe data in abstract form, not in domain-specific form**; from an abstract perspective two unrelated domain datasets can be *equivalent*.

The big picture – one paragraph

The four basic **dataset types** are **tables**, **networks/trees**, **fields**, and **geometry**; other groupings of items are **clusters**, **sets**, and **lists**. They are built from five **data types**: **items**, **attributes**, **links**, **positions**, **grids**. An attribute’s type is **categorical** or **ordered**, with ordered split into **ordinal** and **quantitative**; the ordering direction can be **sequential**, **diverging**, or **cyclic**. Any dataset can be **static** (a file, available all at once) or **dynamic** (a stream).

2.1 Semantics vs Type

Two crosscutting properties must be known about every value:

- **Semantics** – the real-world *meaning* (is this number an age, a postal code, a position?).
- **Type** – the structural / mathematical interpretation (item, link, attribute; table, tree, field; what arithmetic is allowed).

They are independent: knowing the type does not tell you the semantics. Munzner’s example: the row **Basil**, **7**, **S**, **Pear** is meaningless until you are told the semantics (a person: name, age, shirt size, favourite fruit). Information that must be supplied alongside the data is **metadata**.

2.2 The Five Data Types

Data type	Meaning
Item	an individual, discrete entity (a row in a table, a node in a network)
Attribute	a property that can be measured, observed or logged (a column); synonyms: <i>variable</i> , <i>dimension</i>
Link	a relationship between items, typically within a network; synonym: <i>edge</i>
Position	spatial data: a location in 2D or 3D space (e.g. a lat-long pair)
Grid	the strategy for sampling a continuous field (geometric + topological structure of cells)

2.3 The Four Dataset Types (+ Clusters/Sets/Lists)

Dataset type	Built from	Notes
Tables	items, attributes	flat or multidimensional; cells hold values
Networks/Trees	items (nodes), links, attributes	tree = network with hierarchy, no cycles
Fields	grids, positions, attributes	<i>continuous</i> ; cells sampled from a continuous domain
Geometry	items, positions	shape with explicit spatial position; need not have attributes

- **Tables** – rows = items, columns = attributes; each **cell** = (item, attribute) pair holding a **value**.
- **Networks** – items are **nodes** (*vertices*), connected by **links** (*edges*); both nodes and links may carry attributes. A **tree** is the hierarchical special case (each node has one parent, no cycles).
- **Fields** – each **cell** contains a measurement from a **continuous** domain (infinitely many possible values), requiring **sampling** and **interpolation**. Tables/networks are **discrete** by contrast. A **spatial field** samples at spatial positions (e.g. a medical scan). By *number of values per cell*: **scalar** (1), **vector** (2–3, direction+magnitude), **tensor** (many). *SciVis* deals with spatial fields (*position given*); *InfoVis* with nonspatial/abstract data (*position chosen*).
- **Geometry** – pure shape (points, lines/curves, surfaces, volumes); the odd one out because it need not have attributes.

Other groupings of items (not basic dataset types):

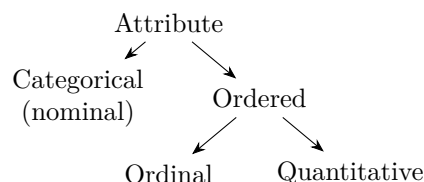
- **Set** – an *unordered* group of items.
- **List** – a group with a *specified ordering* (synonym in CS: *array*).
- **Cluster** – a grouping by *attribute similarity* (items in a cluster are more similar to each other than to items in other clusters).

More complex structures: a **path** (ordered set of segments through a network) and a **compound network** (a network plus an associated tree over its nodes). Real datasets are often hybrid combinations.

2.4 Grid Types (for fields)

Uniform (regular intervals; no need to store geometry/topology) → **rectilinear** (nonuniform spacing) → **structured** (curvilinear shapes, store geometry) → **unstructured** (store both geometry and topology explicitly).

2.5 Attribute Types



- **Categorical** (*nominal*) – no implicit ordering; can only tell *same* (apples) vs *different* (apples vs oranges). E.g. fruit, genres, file types, city names. Any ordering (alphabetical) is external, not inherent.
- **Ordered** – has an implicit ordering. Two sub-types:

- **Ordinal** – well-defined ordering / ranking, but *no full arithmetic*. E.g. shirt size: *large* – *medium* is meaningless, but medium falls between small and large.
- **Quantitative** – measurement of magnitude supporting *arithmetic comparison* (e.g. 68 in – 42 in = 26 in). E.g. height, weight, temperature, counts. (Interval vs ratio is usually collapsed.)

2.5.1 Ordering Direction (for ordered attributes)

- **Sequential** – homogeneous range from a minimum to a maximum (e.g. mountain height from sea level to peak).
- **Diverging** – two sequences meeting at a common *zero/neutral* point (e.g. elevation above/below sea level).
- **Cyclic** – values wrap around to the start (typically time: hour of day, day of week, month of year).

Sketch: Sequential = a single arrow $\rightarrow$; diverging = a double-headed arrow with a block in the middle $\leftarrow \blacksquare \rightarrow$; cyclic = a circular arrow $\circlearrowright$.

Easy point loss

“Ordinal” and “sequential/diverging/cyclic” answer *different* questions. Categorical/ordinal/quantitative is the **attribute type**; sequential/diverging/cyclic is the **ordering direction** and applies *only* to ordered attributes. Do not list “diverging” as an attribute type, and do not call a categorical attribute “sequential”.

2.6 Keys vs Values; Flat vs Multidimensional Tables

- A **key** (*independent* attribute) is an index used to look up **value** (*dependent*) attributes.
- **Flat table** – exactly *one* key; each item is one row. The key may be *explicit* (an ID column) or *implicit* (the row number). Quantitative attributes are usually unsuitable as keys (duplicates not prevented).
- **Multidimensional table** – *multiple* keys are needed to index a cell; the *combination* of all keys must be unique even if a single key has duplicates (e.g. gene $\times$ time $\rightarrow$ activity level).

For **fields** the same idea uses different words: spatial position is a quantitative **key** (independent variable) and the measurement is the **value** (dependent variable). Determining which attributes are keys vs values is often the *outcome* of analysis, not its starting point (recast a flat table into a meaningful multidimensional one).

2.7 Temporal and Time-Varying Semantics

A **temporal** attribute relates to time and has rich, multi-scale hierarchical structure (and possible periodicity). It can be a *key* or a *value*. A dataset is **time-varying** when time is one of the *keys*. A **time-series** is the common special case: an ordered sequence of time–value pairs (a table keyed by time).

2.8 Dataset Availability: Static vs Dynamic/Streaming

Crosscutting all dataset types:

- **Static** (*offline*) – the whole dataset is available at once, as a file. The default assumption.
- **Dynamic** (*online* / **stream**) – data trickles in over the session: items added/deleted, or values changed. Adds complexity to nearly every stage of the vis process.

2.9 Raw Data Issues in Data Tables (exam “List” topic)

When raw data is transformed into clean data tables (first transform of the reference model), the following **raw data issues** must be handled. The lecture’s named list:

Raw data issues – memorise this list

1. **Errors / outliers** – erroneous or extreme values (e.g. an impossible age).
2. **Variable formats** – the same attribute encoded inconsistently (e.g. dates as 16/8 vs Fall vs 24/7; differing units).
3. **Missing data** – empty cells; handle by *dropping* items/attributes or by *imputation* (quantitative: mean/median/mode, regression; categorical: a dedicated “NA” category, logistic regression).
4. **Variable / mixed types** – a column mixing numbers and text, or wrong type assignment.
5. **Table structure** – wrong shape; e.g. a wide adjacency matrix vs a tidy edge/adjacency-list representation (the same data, two structures).

Other commonly cited issues to mention for full marks: **inconsistent units**, **duplicates** (repeated items/rows), **inconsistent units/scales**, and generally **noisy, incomplete, heterogeneous, unstructured** data (the “visualization challenges” list). Beyond cleaning, data transformation also produces **derived values** (e.g. *trade balance* = *exports* – *imports*; a *log transform* to linearise price-vs-carat).

Tidy data

A clean target structure (Wickham): *each variable is a column, each observation is a row, each cell a single value*. This is exactly a well-formed flat data table and resolves most “table structure” and “mixed type” issues.

How this is asked

Frequent question types: (1) “**List** the raw data issues in data tables” – give the five named items plus duplicates/inconsistent units; (2) given a domain dataset, state its **dataset type** and classify each **attribute** as categorical / ordinal / quantitative and its ordering direction; (3) identify the **key(s)** and decide flat vs multidimensional; (4) explain **static vs dynamic** availability; (5) distinguish *type* from *semantics*. A classic trap: the Austrian-states table – “state” is the categorical key, the other four columns (vehicles, accidents, deaths, population) are quantitative values.

3 Visual Encoding: Marks and Channels

The core of the design space of visual encodings is an *orthogonal* combination of two aspects: graphical elements called **marks**, and **visual channels** that control their appearance. Even complex visual encodings can be decomposed and analysed in terms of their mark and channel structure (Munzner, Ch. 5; slides 21–22).

The one-sentence summary

A **mark** is a basic geometric primitive that depicts an item or a link; a **channel** controls the appearance of a mark. The effectiveness of a channel depends on its *type*: **magnitude** (ordered) channels match ordered data, **identity** (categorical) channels match categorical data.

3.1 Marks

A **mark** is a basic graphical element classified by the number of spatial dimensions it requires:

- **Point** – 0D mark. Conveys information only through *position*; because its area is unconstrained, it *can* be size-coded and shape-coded (e.g. a bubble in a bubble chart).
- **Line** – 1D mark. A bar in a bar chart is a line mark. Its length is "taken" by the encoded quantity, so it cannot be size-coded along that same direction, but its *width* can carry a second attribute.
- **Area** – 2D mark. Both size dimensions are intrinsically fixed by its shape, so area marks are typically *not* size- or shape-coded (e.g. a country on a choropleth map). A 3D **volume** mark exists but is rarely used.

Mark types for items vs. links. For tabular data a mark is an **item**. For network data a mark can be a **node** (point/line/area) or a **link**. The two link mark types are: **connection** (a line showing a pairwise relationship) and **containment** (an area enclosing/nesting items to show hierarchy). Links cannot be points; individual items can.

Sketch: three rows of icons – top: a dot (point/0D), a bar (line/1D), a filled blob (area/2D); bottom: two nodes joined by an edge (connection) and a big box with small boxes nested inside (containment).

3.2 Channels and the magnitude vs. identity split

A **visual channel** is a way to control mark appearance, *independent* of mark dimensionality. The human perceptual system has two fundamentally different sensory modalities, which splits all channels into two families:

The fundamental split

Identity / "what-where" channels (psychophysics: *metathetic*) answer "what is it / where is it?" – e.g. shape, color hue, spatial region, motion pattern. They suit **categorical** attributes.

Magnitude / "how-much" channels (psychophysics: *prothetic*) answer "how much is there?" – e.g. length, area, volume, luminance, saturation, tilt, position. They suit **ordered** (ordinal + quantitative) attributes.

List of channels. Spatial: **aligned (common-scale) position**, **unaligned position**, **depth** (3D position), **spatial region**. Color: **hue**, **saturation**, **luminance**. Size (one per added dimension): **length** (1D), **area** (2D), **volume** (3D). Motion: **motion pattern**, **direction**, **velocity**. Plus **angle/tilt**, **curvature**, and **shape** (a complex phenomenon treated as one channel).

3.3 The two guiding principles

Expressiveness & Effectiveness (Mackinlay 1986)

Expressiveness: the encoding should express *all of, and only*, the information in the data. Ordered data must be shown so the perceptual system senses order; unordered data must *not* imply an order that does not exist. $\Rightarrow$ *match channel type to attribute type*.

Effectiveness: the *importance* of an attribute should match the *salience* (noticeability) of its channel. $\Rightarrow$ *most important attribute* $\rightarrow$ *most effective channel*; less important attributes get less effective channels.

Classic expressiveness violation

Encoding an unordered categorical attribute (e.g. country) with an ordered magnitude channel (e.g. luminance or length) implies a ranking that does not exist. This is "a common beginner's mistake." The reverse – categorical hue for a quantitative attribute – destroys perceived order.

In both cases the expressiveness principle is violated even if the chart "works."

3.4 Channel effectiveness rankings

Combining the two principles yields two ranked lists (Munzner Fig. 5.6), most effective at the top. Spatial channels top *both* lists and are the only ones effective for both data types – choosing what to encode with position is the single most central encoding decision, because position-encoded attributes dominate the user’s mental model.

Rank	Magnitude (ordered)	Identity (categorical)
1	Position on common scale (aligned)	Spatial region
2	Position on unaligned scale	Color hue
3	Length (1D size)	Motion
4	Tilt / angle	Shape
5	Area (2D size)	
6	Depth (3D position)	
7	Color luminance	
8	Color saturation	
9	Curvature	
10	Volume (3D size)	

Note: luminance and saturation are roughly equally effective, as are curvature and volume at the bottom.

3.5 Accuracy: Cleveland & McGill and Stevens

Cleveland & McGill (1984) accuracy ranking. Controlled experiments on the magnitude channels gave an empirical ordering of elementary perceptual tasks, from most to least accurate:

1. Position along a common scale (aligned)
2. Position along identical, non-aligned scales (unaligned)
3. Length
4. Angle / slope
5. Area
6. Volume $\approx$ Curvature $\approx$ Luminance (equivalence class)
7. Color hue (*as a magnitude channel – last; very different from its 2nd place as an identity channel*)

Heer & Bostock (2010) replicated this by crowdsourcing; the one discrepancy is that *length and angle judgements turned out roughly equivalent*. These accuracy results are the primary basis for the channel rankings above.

Stevens’ power law. Psychophysics shows that the apparent magnitude S of a sensory channel follows a power function of the physical intensity I :

$$S = I^n.$$

The exponent n depends on the modality and ranges from sublinear 0.5 (brightness) to superlinear 3.5 (electric current).

- $n = 1$ (**length**): perception is accurate – doubling the stimulus doubles the perception.
- $n < 1$ **compressive** (e.g. **area** ≈ 0.7 , **brightness** ≈ 0.5): the channel is perceptually *compressed* $\Rightarrow$ users *underestimate* large values. Doubling brightness looks much less than twice as bright.

- $n > 1$ **expansive** (e.g. red–gray **saturation**, electric current): the channel is *magnified* $\Rightarrow$ users *overestimate*.

Why position beats length – Weber’s Law

Perception is fundamentally *relative*, not absolute (**Weber’s Law**: $\delta I/I = K$). Unframed, unaligned bars are hard to compare; adding a *common frame* or *alignment* gives a common scale, turning a hard length judgement into an easy position judgement. This is why aligned position on a common scale ranks first.

3.6 Channel characteristics

Effectiveness is analysed along five criteria: accuracy, discriminability, separability, popout, and grouping.

Discriminability. How many distinguishable **bins** (steps) does a channel provide? The number of attribute values shown must not exceed the channel’s bin count. Example: **line width** has very few bins – great for 3–4 values, poor for hundreds (it starts looking like an area). If they mismatch, aggregate the attribute or pick another channel.

Separability vs. integrality. Channel pairs lie on a continuum from **separable** (can be attended independently) to **integral** (inextricably fused; the brain perceives an unanticipated combination).

Channel pair	Interference	Result
Position + hue	none (fully separable)	two clean groupings
Size + hue	some	hue hard to read on small marks
Width + height	significant (integral)	fused into perceived <i>area</i> (small/med/large)
Red + green (RGB axes)	major (integral)	perceived as one combined color

Integral is not "bad": horizontal+vertical size is a *good* choice if you want a single attribute with three groups (small, flattened, large). Match the pair’s character to the information.

Popout (preattentive processing). A distinct item "pops out" from distractors in time *independent of the number of distractors* – our low-level visual system processes the channel in massively parallel fashion (also called preattentive / tunable detection). Channels supporting popout include color hue, shape, tilt, size, proximity, motion, shadow direction.

Popout does not combine

Popout works for *one channel at a time*. A red circle does *not* pop out from a field mixing {red, blue} $\times$ {circle, square} – it requires **serial search** (time grows linearly with distractors). A few pairs are exceptions (space+color, motion+shape); never count on popout for three or more channels. Also, parallelism is *not* preattentive.

Grouping. Strength of perceptual grouping cues, strongest first: **containment** (area) > **connection** (line) > **proximity** (spatial region) > **similarity** (hue, motion, carefully chosen shape). This proximity strength is exactly why spatial region is the top categorical channel. Use shape and motion with care: a forward vs. backward "C" does not form a selectable group, while circle vs. star does.

3.7 Channel interference & redundant encoding

Two ideas often confused:

- **Interference** – encoding *different* attributes in non-separable channels (size+hue, width+height) so the user cannot recover each attribute. Avoid.
- **Redundant encoding** – encoding *one* attribute in *two* channels at once (e.g. both hue *and* shape, or both position *and* color) to make it very easily perceived. The cost is that channels are "used up," so fewer total attributes can be shown.

How this is examined

Visualization Critique (open question). Given a chart, you must:

- Decompose it into *marks* and *channels* per attribute.
- Check *expressiveness*: is each attribute's channel type matched to its data type? Flag categorical-on-magnitude (false order) or vice versa.
- Check *effectiveness*: is the most important attribute on the most effective channel (position > length > angle > area > color)?
- Invoke *Stevens' power law* to argue over/underestimation: area ($n \approx 0.7$) and brightness ($n \approx 0.5$) are underestimated; saturation is overestimated; length ($n = 1$) is accurate.
- Spot *channel interference* (e.g. width vs. height read as area; small marks make hue unreadable) and suggest separable alternatives or redundant encoding.

MCQ on channel properties. Be ready to: classify a channel as magnitude vs. identity; order channels by effectiveness; recall the Cleveland & McGill ranking; identify separable vs. integral pairs; state which channels support popout; and distinguish redundant encoding from interference.

Common confusions to avoid

Hue is *2nd* as an identity channel but *last* as a magnitude channel. "Position on a common scale" $\neq$ "length" – the frame/alignment is what makes position more accurate (Weber's Law). Effectiveness = importance $\rightarrow$ salience; expressiveness = data type $\rightarrow$ channel type. Do not mix them up.

4 Perception and Color

Visualization works because it offloads cognition onto the human **visual system**, a high-bandwidth, partly pre-cognitive channel. Knowing how the eye and early vision behave tells us *which* visual channels are fast, accurate and orderable, and which ones (notably hue) are not. This section grounds the design rules used elsewhere (expressiveness, effectiveness, channel ranking) in perception, and then covers color, the single most over-used and most-often-misused channel in the exam's "Visualization Critique" questions.

Why perception matters for vis

A good encoding lets the visual system answer the task *pre-attentively* (in a glance, ~ 200 ms) instead of forcing slow serial reading. Bad encodings (rainbow color, 3D bars, hue for ordered data) fight the visual system and force slow, error-prone interpretation.

4.1 The Human Visual System

4.1.1 Eye, retina, rods and cones

Light from the scene is projected by the cornea/lens onto the **retina** at the back of each eye. The retina is covered with photoreceptors:

- **Cones** – color-sensitive (three types, tuned to long / medium / short wavelengths), concentrated in the centre of the retina; responsible for detailed, color daytime vision.
- **Rods** – highly light-sensitive but *not* color-sensitive; dominate peripheral and low-light vision.

4.1.2 Fovea and acuity

The **fovea** is the small central region of the retina where image detail and color rendition are best (densest cone packing). Crucially, the fovea covers a viewing angle of only about 2° – roughly a thumbnail held at arm’s length. Moving outward through the **parafovea** to the **periphery**, **visual acuity** (the ability to resolve detail) *falls off dramatically* [Spence, Sec. 4.1].

4.1.3 Fixations and saccades

Because only a tiny region is seen in detail, the eye constantly jumps around the scene. Each **fixation** (gaze held on one point) lasts about **200 ms**; fixations are connected by very fast (~ 20 ms) **saccades**. Viewing a whole image involves ~ 100 fixations. **Overt attention** is paid to a few fixated points; **covert attention** to parafoveal/peripheral items helps decide where to look next.

4.1.4 Visual cortex (it’s complicated)

Retinal signals pass through the LGN to the primary visual cortex **V1** (the main entry point; neurons highly selective for gradients, i.e. “edge detection”), then V2/V4 with increasingly complex response, up to the inferior temporal cortex (object recognition, memory). Exam-relevant takeaway: *luminance contrast drives edge detection*, which is why luminance is a good channel for fine spatial detail and hue is not.

Sketch: Retina disk with a small bright dot labelled *fovea* (2°), a ring *parafovea*, and an outer ring *periphery*; an arrow “acuity” pointing outward and shrinking. Beside it: a zig-zag gaze path (fixations ≈ 200 ms, saccades ≈ 20 ms).

4.2 Preattentive Processing and Popout

A **limited** set of visual channels is detected *rapidly and in parallel*, within ~ 200 – 250 ms, before focused attention engages.

Operational definition (exam-quotable)

If finding a target takes a **fixed amount of time regardless of the number of distractors**, the channel is processed **preattentively**. The target then “pops out”.

A single red dot among blue dots pops out instantly whether there are 12 or 50 distractors. Channels that support popout include color (hue), orientation (tilt), size, curvature, motion, and others (Healey’s catalogue).

Conjunction search – the key limitation

Popout works only when the target differs in *one* unique visual property. If the target shares each of its properties with some distractors and is defined only by a **combination** of features (e.g. “the red *circle*” among red squares and blue circles), there is no unique channel to search for. This is a **conjunction search: not possible pre-attentively**, so a slow *serial* search is required and search time grows with item count.

Important nuance: a conjunction of *spatial position* with one other channel (color or shape) *can* still be searched preattentively – e.g. in a scatterplot the conjunction of space $\times$ color, or space $\times$ shape, pops out. The hard case is conjunctions of two non-spatial channels.

How this is asked

“Why is it slow to find X in this chart?” $\Rightarrow$ identify whether the target has a unique preattentive feature or requires conjunction search. Design fix: encode the target’s category with a *single* popout channel (e.g. make the outliers a distinct hue) so they pop out preattentively.

4.3 Gestalt Laws of Grouping

Gestalt laws describe how we perceive **patterns** / **groupings** in visual displays, and translate directly into design principles for information displays.

Law	We group items that...	Vis example
Proximity	are close together in space	clusters in a scatterplot read as groups
Similarity	look alike (same hue/shape/size)	same-color points = same category
Connectedness	are joined by lines	node-link edges, slope graphs
Continuity	lie along a smooth continuous contour	a line is followed through crossings
Closure	form a closed/IMPLIED contour	we “complete” a shape
Symmetry	are symmetric about an axis	mirrored layouts read as one object
Common fate	move together	animated points moving together group
Common region	lie inside a shared enclosure	a shaded hull / bounding box

Which is strongest?

Connectedness (and shared enclosure / common region) can be a *more powerful* grouping cue than **proximity**: items joined by a line or surrounded by a hull are grouped even when other cues (color or distance) would group them differently. The slides state explicitly: “*connectedness can be a more powerful grouping principle than proximity.*”

Sketch: Two scatter clusters. (a) proximity alone $\rightarrow$ two groups by distance. (b) draw a connecting line / shaded blob that links one point from the left cluster to the right cluster $\rightarrow$ it now reads as belonging to the enclosed group, overriding proximity.

4.4 Weber’s Law and the Just Noticeable Difference

4.4.1 JND

The **Just Noticeable Difference (JND)** is the amount a stimulus must be changed to be detectable by human observers *at least half of the time*. It is determined *empirically*.

4.4.2 Weber’s law

The JND is *not* an absolute amount: it is proportional to the background stimulus magnitude. To perceive a difference between a background intensity I and the background plus stimulation ΔI , the difference must be proportional to the background:

$$k = \frac{\Delta I}{I} \iff \Delta I = k I, \quad k = \text{const (Weber fraction)}.$$

Implication: we judge *relative*, not absolute, differences. Adding 5 dots to a field of 10 is obvious; adding 5 to a field of 100 is not, even though the absolute change is identical.

Consequences for design

- Integrating Weber’s law over stimulus intensity gives **Fechner’s law**: perceived intensity is *logarithmic*, $p = k \ln(I/I_0)$, where I_0 is the absolute threshold.
- More generally, **Stevens’ power law** models perceived sensation as $p = k I^a$ (a a measured exponent per channel). Length $a \approx 1$ (perceived $\approx$ veridical); area $a \approx 0.7$ and brightness $a \approx 0.5$ are *compressed* (under-estimated), while saturation and electric shock are exaggerated. This is the perceptual basis for why *position/length beats area/color* in the channel ranking.
- Practical rule: length differences are most accurately judged when bars share a **common baseline and are adjacent**; offset or stacked bars make comparison much harder.

4.5 Color

4.5.1 Color is three channels

Colloquial “color” is best understood as **three separate channels**: **luminance** (light–dark), **hue** (red, green, blue, ...), and **saturation** (vividness). The major colormap design choice is which of these to use for what.

Ordered vs. unordered channels (Munzner)

- **Luminance**: a *magnitude* (ordered) channel – people reliably order grays, always placing gray between black and white.
- **Saturation**: ordered, but *less easily* (only ~ 3 – 4 discriminable steps for small marks).
- **Hue**: an *identity* (categorical, **unordered**) channel. Asked to order red/blue/green/yellow patches, people disagree. **Hue cannot be ordered** – learned conventions (rainbow, traffic lights) are higher-level, not perceptual.

4.5.2 Opponent color theory

The retina’s signals are immediately processed into **three opponent color channels**: red–green, blue–yellow, and a black–white channel encoding **luminance**. The luminance channel carries *high-resolution* edge information; the two chromatic channels are *lower resolution*. This is why luminance (not hue) should carry fine spatial detail.

Opponent theory also explains **color blindness** (more accurately *color vision deficiency*), affecting ~ 8 – 10% of men in its common form: the **red–green** channel is reduced/absent while blue–yellow still works $\Rightarrow$ **avoid red–green encodings**.

4.5.3 Colormap types and when to use each

The colormap taxonomy mirrors the data-type taxonomy.

Colormap	Data	Use / rule
Categorical (qualitative)	categorical	Distinct hues for unordered groups; <i>segmented</i> . Hue = identity channel. Limited to ~ 6–12 discriminable bins (12 max). Requires a legend .
Sequential	ordered (ord./quant.)	Progression from min to max along an ordered channel (luminance, possibly with one hue), e.g. light $\rightarrow$ dark pink.

Colormap	Data	Use / rule
Diverging	ordered with meaningful midpoint	Two sequential ramps meeting at a neutral midpoint (zero / mean), e.g. blue–white–red “warming stripes”. Use when a zero/reference point is meaningful (gain vs. loss).
Bivariate	two attributes	Encodes two attributes at once. Easy to read when the 2nd attribute is <i>binary</i> ; quickly unreadable when both have several levels.

Colormaps can be **continuous** (good for quantitative / spatial fields) or **segmented**/binned (good for categorical; emphasizes discrete bins).

Rules for categorical color

- Use $\leq 6\text{--}12$ colors; more than 12 hues are very hard to discriminate (Ware). Count the background and default object colors (black/white/gray) in your budget.
- Prefer **easily nameable** colors (only ~ 8 are consistently named): start with the opponent-axis colors red/green/blue/yellow, then orange/brown/pink/purple/cyan.
- Match saturation to mark size: *small* marks (points, lines) $\rightarrow$ highly saturated; *large* areas $\rightarrow$ low saturation/pastel.
- If categories exceed the color budget, aggregate into groups or an “other” bin, or switch to an additional channel (shape, size).
- Categorical color always needs a **legend**.

4.5.4 The rainbow / jet colormap problem

Using the rainbow (“jet”) colormap for *ordered* data is a classic mistake. Munzner lists three serious perceptual problems:

Three problems with the rainbow colormap

1. **Hue is used to indicate order**, but hue is an identity channel with *no implicit perceptual ordering* (expressiveness mismatch).
2. **Not perceptually uniform**: equal steps in data are *not* perceived as equal – some ranges show several distinct colors, others look uniform (a wide band that just “looks yellow”).
3. **Fine detail is lost**: hue cannot convey fine-grained detail; luminance would, because luminance contrast drives edge detection.

Add a fourth, practical one: rainbow is **not color-blind safe** (red/green confusions).

4.5.5 Perceptually uniform maps (viridis)

The fixes: (i) design **monotonically increasing luminance** colormaps so that order is carried by the magnitude (luminance) channel while hue only marks semantic regions; (ii) use a **perceptually uniform** colormap such as **viridis** – built in a perceptually uniform space (cf. CIE $L^*a^*b^*$) so equal data steps look equally different, with monotonic luminance, and it is color-blind friendly. (A perceptually linear rainbow is possible but loses brightness/dynamic range, so it is rarely used.)

Color critique checklist

For any color encoding, ask: (1) Is the data *ordered*? If yes, do *not* use hue alone – use luminance / a sequential or diverging map. (2) Is there a meaningful midpoint? $\rightarrow$ diverging. (3) Categorical with ≤ 12 groups? $\rightarrow$ distinct nameable hues + legend. (4) Is it color-blind safe (no red–green dependence; redundant encoding)? (5) Is it perceptually uniform (avoid

rainbow/jet; prefer viridis)?

color $\neq$ color

“Color” is not one thing. Saying “encode this with color” is too vague: state *which* channel (luminance/hue/saturation) and check it matches the attribute type (ordered $\rightarrow$ luminance; categorical $\rightarrow$ hue).

5 Design Principles and Visualization Critique

This section is the toolkit for the exam’s recurring “*What is wrong with this visualization? Suggest an improvement*” questions. It collects Tufte-style integrity rules and Munzner’s encoding rules, then gives a reusable **critique checklist** and two fully worked critiques (a rainbow choropleth and area-encoded circles). The golden rule underneath everything: **function before form** – start from the task (*what* data, *why* task), then choose the encoding; never start from a pretty picture.

Two governing principles (Munzner)

- **Expressiveness:** the visual structure should encode *all and only* the information in the data. Ordered data must look ordered; unordered data must *not* look ordered.
- **Effectiveness:** the most important attributes should get the most effective channels (faster to read, more distinctions, fewer errors). Spatial channels (position) are most effective.

5.1 Data–Ink Ratio and Chartjunk (Tufte)

Data–ink is the ink that encodes actual data. Tufte’s **data–ink ratio** is

$$\text{data–ink ratio} = \frac{\text{ink used to show data}}{\text{total ink used}}.$$

Chartjunk is non-data ink that adds no information: heavy gridlines, 3D shading, decorative backgrounds, redundant frames, moiré patterns.

Rules

Maximize the data–ink ratio (within reason) and **erase chartjunk**: remove or mute gridlines, drop 3D effects and shadows, avoid decorative clipart. Every drop of ink should earn its place by conveying data.

5.2 The Lie Factor (graphical integrity)

Tufte’s **lie factor** measures whether the visual effect matches the data:

$$\text{lie factor} = \frac{\text{size of effect shown in the graphic}}{\text{size of effect in the data}}.$$

“Size of effect” = (later value – earlier value) / earlier value, the relative change. A lie factor of **1** is honest; > 1 exaggerates, < 1 understates. Tufte considers 0.95–1.05 acceptable.

Worked computation

A quantity grows from 18 to 27 in the data (effect = $(27 - 18)/18 = 0.50$, i.e. +50%). The graphic draws bars whose *heights* go from 1 cm to 4 cm (shown effect = $(4 - 1)/1 = 3.0$, i.e. +300%). Then

$$\text{lie factor} = \frac{3.0}{0.5} = 6.0,$$

so the graphic exaggerates the change six-fold (a classic truncated-axis or area-scaling lie).

Common lie-factor sources

Truncated / non-zero baselines on bar charts; encoding a 1D value as 2D *area* (or 3D volume); inconsistent axis scaling. Always check the baseline starts at zero for bars.

5.3 Over-Counting via Area (1D value as 2D size)

When a single quantity is mapped to a circle, designers often scale the *radius* (or diameter) by the value. But the perceived magnitude is the **area**, which then grows with the *square* of the value: double the value $\Rightarrow$ radius $\times 2 \Rightarrow$ area $\times 4$. The graphic over-states the difference fourfold.

The “men of talent” circle example

Tufte’s classic case: circles sized to show a quantity were scaled by diameter, so a value twice as large was drawn with $4\times$ the area – a huge, misleading exaggeration. **Rule:** if you must use circles, scale by *area* (value $\propto$ area, so radius $\propto \sqrt{\text{value}}$). Even then, *area is perceptually under-estimated* (Stevens exponent ≈ 0.7), so length/position bars are a better encoding for magnitude.

Area perception cuts both ways

- **Mis-encoding** (radius scaling) over-states differences ($\times \text{value}^2$).
- **Perception** of correctly area-scaled marks *under-states* differences (compression, exponent < 1).

So area is doubly problematic: prefer length on a common scale for quantities.

5.4 Ordering of Bars, Axes and Categories

For *unordered categorical* keys, the layout order is a free design choice – use it to carry information.

Order by value, not by alphabet

Sort bars / rows / categories by their **data value** (e.g. descending), not alphabetically or by arbitrary ID. A value-sorted bar chart makes rank, extremes and the distribution shape readable at a glance; an alphabetical order hides them. For ordinal keys, keep the intrinsic order.

5.5 Expressiveness Violations

The most common beginner mistake

Showing ordered data with an unordered (identity) channel, or implying an order that does not exist.

- Quantitative value on *hue* (rainbow colormap) – hue is an identity channel; it implies categories, not order.
- Categorical attribute on *position on a continuous axis* – implies an ordering the data does not have.

Fix: ordered attribute $\rightarrow$ magnitude channel (position, length, luminance); categorical attribute $\rightarrow$ identity channel (hue, shape, spatial region).

5.6 Effectiveness, Stevens’ Power Law and the Channel Ranking

Effectiveness is grounded in perception (Stevens’ power law: perceived sensation $p = k I^a$; see the Perception section). Channels are split into **magnitude channels** (for ordered attributes) and

identity channels (for categorical attributes), each ranked by accuracy.

Magnitude (ordered), best→worst	Identity (categorical), best→worst
Position on common scale	Spatial region
Position on unaligned scale	Color hue
Length (1D size)	Motion
Tilt / angle	Shape
Area (2D size)	
Depth (3D position)	
Color luminance, saturation	
Curvature	
Volume (3D size)	

Takeaways

Spatial (position) channels are the most effective for both ordered and categorical data. Put the most important attribute on position. Area, color and especially volume are weak for magnitude.

5.7 Channel Interference (separable vs. integral)

When two attributes are encoded on two channels, the channels should be **separable** – selectively attendable – so each can be read independently. **Integral** channel pairs fuse perceptually and cannot be separated.

Separable (good)	Integral (bad for 2 attributes)
Position + hue (fully separable)	Red + green (major interference)
	Width + height (size dimensions fuse)
	Size + hue (some interference)

Channel interference

Encoding two *separate* attributes in an integral pair (e.g. red–green, or x-size + y-size) fails – viewers see one fused blob, not two attributes. Choose channels with as little interference as possible.

5.8 Eyes Beat Memory: Juxtaposition vs. Animation

Eyes beat memory

Comparison is far more accurate when both items are visible *at the same time* (**juxtaposition** – side-by-side small multiples) than when the viewer must hold one in memory while looking at the other. **Animation forces comparison through memory** and is therefore weak for precise comparison; prefer side-by-side static views. (Redundant encoding – using > 1 channel for the *same* attribute – is the opposite, and is fine: it aids robustness, e.g. color + symbol for color-blind safety.)

5.9 Problems with 3D

Unjustified 3D

Adding a third spatial dimension to data that is not inherently 3D causes:

- **Perspective distortion** – equal data values map to unequal screen sizes depending on depth, so comparison is biased.
- **Occlusion** – nearer marks hide farther ones; data is lost.
- **Shadows / shading** and reduced **text legibility**.

Slide rule: “*No unjustified 3D!*” A 3D bar chart is almost always worse than its 2D version.

When 3D is justified

Use 3D only for **inherently spatial** data where the third dimension *is* part of the data – e.g. 3D medical/volume data, molecular structures, terrain, fluid-flow fields. There the benefit of matching the data’s spatial nature outweighs the distortion/occlusion costs (and interaction such as rotation mitigates them).

5.10 Function Before Form

Process: What / Why / How

Design follows the loop **What** (data abstraction) → **Why** (task abstraction) → **How** (visual encoding + interaction). Decide the task *first*; the most expressive and effective encoding follows from it. A chart that looks impressive but does not support the task fails.

5.11 The Critique Checklist (use on any chart)

Step-by-step critique

1. **What & Why**: what is the data type of each attribute (categorical / ordered / quantitative) and what is the task (identify / compare / summarize, trend / outlier / correlation)?
2. **Expressiveness**: does each channel match its attribute type? (ordered→magnitude, categorical→identity). Any false ordering?
3. **Effectiveness**: is the most important attribute on the most effective channel (position/length), not on a weak one (area/hue)?
4. **Integrity**: zero baseline? lie factor ≈ 1 ? 1D value not encoded as 2D area? axes consistent?
5. **Color**: ordered data not on hue (no rainbow); diverging needs a midpoint; categorical ≤ 12 nameable hues + legend; color-blind safe (no red–green); perceptually uniform (viridis)?
6. **Channels**: separable, not integral? not too many channels at once?
7. **Ordering**: bars/categories sorted by value, not alphabet?
8. **Chartjunk & 3D**: remove non-data ink; no unjustified 3D.
9. **Comparison**: juxtaposed (eyes), not animated (memory)?
10. **Then suggest a concrete fix** for each issue.

5.12 Worked Critique 1: Rainbow Choropleth (Texas map)

Scenario: a US state/county choropleth coloring a continuous quantitative attribute (e.g. a per-county ratio) with the **rainbow** / **jet** colormap.

Critique

- **Expressiveness violation:** the value is *ordered* but it is encoded with *hue*, an unordered identity channel – the legend order (pink, blue, green, yellow, orange...) carries no perceptual order, so readers cannot tell which county is higher without constant legend lookups.
- **Not perceptually uniform:** equal value steps look unequal; some adjacent bins look identical while others jump.
- **Fine detail lost:** hue cannot resolve fine differences; luminance contrast (needed for edges) is missing.
- **Not color-blind safe:** red/green bins are confusable for ~8–10% of men.

Fix

Replace with a **sequential** colormap that uses *luminance* (light → dark, single or few hues), or a **perceptually uniform** map (**viridis**); if there is a meaningful midpoint, use a **diverging** map with a neutral centre. Use a **color-blind-safe** palette (ColorBrewer), and keep the legend ordered by value.

5.13 Worked Critique 2: Area-Encoded Circles

Scenario: two (or more) quantities shown as circles, with the *radius/diameter* proportional to the value (“men of talent” style).

Critique

- **Over-counting via area:** radius $\propto$ value makes *area* $\propto$ value², so the bigger circle looks far larger than the data warrants (lie factor > 1).
- Even with correct area scaling, **area is under-estimated** perceptually (Stevens exponent ≈ 0.7): differences are hard to judge accurately.
- Comparing *areas* of separated circles is a weak perceptual task (low on the channel ranking).

Fix

Encode the magnitude with **length / position on a common scale** – a **bar chart with a zero baseline**, bars adjacent and sorted by value. This puts the value on the most effective channel, removes the squaring error, and lets readers compare precisely. If circle sizing is unavoidable (e.g. a bubble map), scale by *area* (radius $\propto \sqrt{\text{value}}$) and add direct value labels.

Phrasing your answer

A full-credit critique names the *principle* (e.g. “expressiveness violation: ordered data on hue”), explains the *perceptual reason*, and gives a *concrete fix* (“use a sequential luminance/viridis map”). Bullet each issue; do not just say “it looks bad”.

6 Tasks and Interaction

This section covers the *why* of visualization (Munzner’s task abstraction, ch. 3) and the *how* of interaction (selection, brushing & linking, overview+detail, focus+context, dynamic queries, coordinated multiple views, high-dimensional brushing after Ward, and quantitative visual analytics). The unifying idea from the lecture: **interaction is a cognitive amplifier**. Static visualization fails for large data, multiple questions, and evolving hypotheses; interaction lets one display answer many questions over time.

The what–why–how chain (Munzner)

Every analysis instance is described by three orthogonal questions:

- **What** is shown – the data abstraction (dataset and attribute types).
- **Why** is it used – the *task* abstraction: **actions** (verbs) on **targets** (nouns).
- **How** is it built – the idiom: encode, manipulate, facet, reduce.

Key principle: **why does not dictate how** – the same goal can be served by many idioms. Tasks are stated in *abstract* (domain-independent) terms so that two domain tasks ("contrast patients..." vs. "see if results match...") are recognised as the same abstract task ("compare values between two groups"). Complex activities are a **chained sequence** of tasks: the output of one becomes the input of the next (e.g. a *discover* session feeds a later *present* session).

6.1 Task Abstraction: Actions (Munzner ch. 3)

Actions sit at three independent levels; usually you specify all three.

6.1.1 High level – Analyze (consume vs. produce)

- **Consume** existing information:
 - **Discover** – find new knowledge; *generate* or *verify* a hypothesis (the classic motivation for sophisticated interactive idioms; close to *explore*).
 - **Present** – communicate something already understood to an audience (telling a story; decision support, teaching). Knowledge is known to the presenter in advance.
 - **Enjoy** – casual encounters driven by curiosity (e.g. Name Voyager).
- **Produce** new material for later use:
 - **Annotate** – add graphical/textual notes to existing elements (an annotation tied to data items can be seen as a new attribute).
 - **Record** – save persistent artifacts (screenshots, bookmarks, parameter settings, interaction logs, a branching *graphical history*); supports analytic provenance.
 - **Derive** – compute new data from existing data (a.k.a. *transform*). The data abstraction is an *active design choice*: "don't just draw what you're given – decide the right thing to show and create it." E.g. plot $trade\ balance = exports - imports$ directly instead of two curves.

6.1.2 Mid level – Search (a 2×2 of known/unknown)

Classified by whether the *target* (identity, the *what*) and the *location* (the *where*) are known.

	Target known	Target unknown
Location known	Lookup	Browse
Location unknown	Locate	Explore

Lookup: know what and where (look up "humans" in a known spot of a tree). *Locate*: know what, not where (hunt for "rabbits"). *Browse*: know roughly where, not exactly what (scan a subtree / a date column). *Explore*: know neither – start from an overview, hunt for outliers/anomalies. (The verb **find** is the generic synonym for a successful search.)

6.1.3 Low level – Query (scope: one / some / all)

Once targets are found, query them at one of three scopes of increasing breadth:

- **Identify** – one target (return its characteristics, e.g. one US state's result).
- **Compare** – some targets (multiple; typically harder, needs richer idioms).
- **Summarize** – all targets. Synonym **overview**, extremely common in vis.

6.2 Task Abstraction: Targets

Targets are the aspect of the data of interest (nouns).

- **All data: trends** (a.k.a. *patterns*: increases, peaks, plateaus), **outliers** (anomalies, deviants, surprises), **features** (task-dependent structures).
- **One attribute**: an individual value, the **extremes** (min/max), or the whole **distribution**.
- **Multiple attributes: dependency** (one depends on another), **correlation** (tendency to co-vary), **similarity** (quantitative closeness across all values).
- **Network data: topology** (structure of interconnections) and **paths** between nodes.
- **Spatial data: shape**.

Sketch: Draw the Actions tree (Analyze→Consume/Produce; Search 2×2 table; Query: identify/compare/summarize) next to the Targets list (all-data / one-attr / many-attr / network / spatial). This single diagram answers most "name the task" exam questions.

6.3 Interaction Techniques

A useful working definition (Card et al.): "The use of computer-supported, *interactive*, visual representations of abstract data to amplify cognition." Why interact: encoding determines which questions are easy, and there is **no single best visualization** – interaction lets the user switch encodings and ask new questions on the fly.

Catalogue of techniques

Change visual encoding: change arrangement (e.g. re-baseline a stacked bar chart for easier sub-section comparison), change marks/channels, change shown dimensions, reorder/sort.

Selection & highlighting: mark items as interesting; highlight makes them stand out (color/emphasis). The most fundamental brush operation.

Brushing & linking: select in one view → *linked highlighting* in all views.

Navigation: pan/scroll and **zoom** (geometric vs. semantic).

Details-on-demand: show full record for an item when requested.

Dynamic queries / filtering: show items conditionally; interactive query refinement.

6.3.1 Shneiderman's Visual Information-Seeking Mantra

Shneiderman 1996

"Overview first, zoom and filter, then details on demand."

- **Overview** – starting point; shows overall patterns; needed for navigation/orientation.
- **Zoom & filter** – narrow to a region of interest, hide irrelevant items.
- **Details on demand** – inspect individual cases/variables only when asked.

6.3.2 Zoom: geometric vs. semantic; pan

Geometric zoom simply scales the view. **Semantic zoom** changes *what* is shown / how items are rendered as scale changes (more or different content at finer levels), not just size. **Pan/scroll** provides a larger virtual screen but is clunky and shows only one piece at a time. Rule of thumb: a long pan is bad – "zoom out, pan a little, zoom in."

6.3.3 Overview+Detail vs. Focus+Context

The **core problem**: large data needs both *overview* (patterns) and *detail* (inspection), but not both at full resolution at once. Zoom-in loses context, zoom-out loses detail, pan causes disorientation.

- **Overview+detail** – combine via *space* (different screen regions for overview and detail) or via *time* (alternate sequentially in the same place).
- **Focus+context (F+C)** – integrate a (zoomed) focus and its context within *one* view; "seeing detail without losing the big picture." Focus = detail, context = structure.

F+C design space (three families).

- **Distortion-oriented** – bend space; like a *fisheye lens*: center enlarged, periphery compressed, everything stays visible but not equally emphasized (e.g. Perspective Wall).
- **Views-/layers-based** – separate views for focus and context (e.g. a small overview map / minimap, Macroscope).
- **In-place** – no geometric distortion; emphasize via color, blur, transparency (e.g. Semantic Depth of Field, SDOF).

Fisheye terminology (Furnas). A fisheye view is computed from:

- **Focal point** – the item/coordinate the user focuses on.
- **Level of Detail (LoD)** – intrinsic a-priori importance of each data element.
- **Distance from focus** – how far an item is from the focal point.
- **Degree-of-Interest (DOI) function** – governs how each item is rendered, e.g.

$$\text{DOI}(\text{item}) = \text{LoD}(\text{item}) - \text{Distance}(\text{item}, \text{focus}).$$

DOI can be *continuous* (smooth interpolation), *filtering* (items past a threshold disappear), *stepped* (discrete regions), or *semantic* (rendering changes type at different levels).

Why F+C matters / trade-offs

Benefits: maintains orientation, reduces navigation effort, supports exploration; aligns well with the information-seeking mantra and data drill-down; also usable for presentation (SDOF). *Requires interaction* (focusing, navigation) with smooth changes – it is dynamic, not static.
Trade-offs: distortion → spatial confusion; layers → split attention; in-place → limited expressiveness.

6.3.4 Response-time budget

≤ 100 ms feels *instantaneous*; ≤ 1 s feels like *free interaction*; > 10 s loses the user's attention entirely (Card, Robertson & Mackinlay 1991). Justifies why interactive idioms must keep latency low.

6.3.5 Seven categories of interaction (Yi et al. 2007)

A complementary taxonomy phrased as user intents: **Select** ("mark as interesting"), **Explore** ("show something else"), **Reconfigure** ("different arrangement"), **Encode** ("different representation"), **Abstract/Elaborate** ("more or less detail"), **Filter** ("show conditionally"), **Connect** ("show related items").

6.4 Linked Views / Coordinated Multiple Views (CMV)

Why linking is crucial

A single 2D view cannot show all attributes of a multidimensional table. CMV shows *various attributes simultaneously in several simple 2D views* (parallel coordinates, scatterplot matrix, histograms, box plots, ...). **Brushing & linking** ties them together: **select items in one view and they are highlighted in all views**. This reveals how a feature in one attribute relates to others – e.g. which items are extreme in *X* and where they fall in *Y, Z, ...* Interaction

(not a fancy single chart) is what makes multidimensional analysis tractable.

Open questions raised in the lecture for CMV/brushing: brushing *reproducibility*, communicating results, keeping focus on analysis (not on fiddling with the interaction), whether brushing is always best, and how to improve it.

6.5 High-Dimensional Brushing (Ward, XmdvTool)

Traditionally brushes live in *screen space* (paint/rubber-band a 2D region). Ward's contribution: **N-dimensional brushes defined in data space** rather than screen space, integrated into XmdvTool (which shows the same multivariate data via scatterplot matrices, star glyphs, parallel coordinates, and dimensional stacking).

Brush specification & operations

Specification (redundant on purpose): direct manipulation (drag/resize the brush boxes per display) or indirect via the *brush toolbox* (global resize: max/half/ $\pm 10\%$).

Brush structure: a brush is a *hyperbox* – an extent (start..end) in each of the N dimensions.

Per-display manipulation: scatterplot (drag boxes in each plot of the $N \times N$ matrix), parallel coordinates (sliders on each axis), glyph and dimensional-stacking brush tools.

Brush operations: highlighting, linking (default across all views), masking/deleting, moving average, quantitative (numeric) display.

Data-driven brushing ("painting"). Instead of setting bounds by hand in every dimension, the user *paints* over data points of interest (shift+drag); a brush hyperbox is generated as the min/max along each axis of the painted points – the *data* drives the brush. Useful when bounds are unknown in dense data. (Demand-driven brushing = the user specifies criteria, e.g. high in dim 1, low in dim 2, any in dim 3.)

Compound / composite brushes. Multiple brushes (XmdvTool supports four) combined with **boolean operators** AND, OR, XOR, NOT. With ramped (non-binary) coverage the operators become arithmetic on coverage $C \in [0, 1]$:

$$C_{\text{OR}} = \max(C_A, C_B), \quad C_{\text{AND}} = \min(C_A, C_B), \quad C_{\text{XOR}} = 1 - |1 - (C_A + C_B)|.$$

Brush boundaries / extents. Containment is usually binary (*stepped* boundary). A **ramped** boundary linearly drops coverage from 1 (inside, at the inner boundary) to 0 (outside) between an inner and outer boundary – distinguishing data near the brush center from data near its edge.

Structure-based brushing. Brushing on a *hierarchical/structural* representation of the data (e.g. a cluster tree) rather than on raw value extents – select by structure, not just by per-dimension ranges.

Brushing exam traps

Do not say a Ward brush is a screen-space rectangle – its key novelty is that it is an *N-dimensional hyperbox in data space*. Do not confuse *data-driven* (paint the points, bounds inferred) with *demand-driven* (state the criteria up front). AND = min, OR = max for ramped coverage (not the other way round).

6.6 Quantitative Visual Analytics (Radoš et al. 2016)

The additional deck pushes interactive analysis from qualitative toward **reproducible and quantitative VA**, via four extension groups:

- **Structured brushing** – brushing characterised along three axes: *brush anchoring*, *brush extent*, *brush movement*, each *unconstrained*, *constrained* (snapped to a grid of percentile cells), or *automatic* (programmatic, reproducible) – improving reproducibility.
- **Percentile brushes** – circle/rectangle and *Mahalanobis* brushes; the Mahalanobis brush takes the underlying data *distribution* / *covariance* into account so a single click+drag matches an elongated cluster.
- **Linked statistics** – overlay numeric summaries (mean, median, midrange, spread) directly on the view and in numerical tables, linking data to its statistics.
- **Change visualization** – traces/graphs of how statistics evolve while refining a brush, plus a difference-plot; supports iterative **drill-down** and *query refinement*.

Underlying tension highlighted in class: *complex interaction + simple data* vs. *simple interaction + complex (pre-derived/aggregated) data* – data derivation/aggregation can be moved into the system (on-the-fly data generation).

How this is asked

- **"Justify the interaction design"** (strategy questions): map each chosen technique to a task. State the *why* (action + target, e.g. *explore* for *outliers*; *compare distributions*), then pick the *how* (overview first → filter → details-on-demand; F+C to keep orientation; CMV with linked highlighting). Remember *why does not dictate how* and there is no single best vis – argue trade-offs and the response-time budget (< 1 s).
- **"Map the data items to a task"**: classify with Munzner – which *action* (lookup/locate/browse/explore × identify/compare/summarize) on which *target* (trend/outlier/distribution/correlation/topology/path/shape).
- **Brushing & linking**: explain selecting in one view highlights the same items everywhere, and *why* that is essential for multidimensional tables (relate a feature in one attribute to all others). For Ward: N-D hyperbox in data space, painting (data-driven), AND/OR/XOR/NOT composite brushes, ramped vs. stepped extents, structure-based brushing.
- **Focus+context**: name the three families (distortion / views-layers / in-place), the fisheye DOI = LoD – distance, and the trade-offs (distortion → confusion, layers → split attention).

7 High-Dimensional Data Visualization

High-dimensional data has **many attributes** (columns / dimensions / features / variables) per item. A working definition from the lecture: data are **high-dimensional** when our traditional visual encodings for tabular data *break* (Yang et al., 2003) – typically already from dozens of dimensions on. Structured data has named, interpretable attributes (e.g. demographic indicators, gene expression); unstructured data (text, images, audio) is turned *into* a high-dimensional feature matrix via bag-of-words / document-term matrices, flattened pixels (MNIST → 784-dim vector), word embeddings (word2vec) or learned latent codes (autoencoders, CNNs). Abstractly everything is a flat $n \times m$ table X (n items × m attributes), only m is very large and the attributes may not be interpretable.

Two fundamental strategies

- **Feature selection**: pick a *subset* of the existing dimensions, *without* transforming them. Axes stay interpretable. Dimensions (or dimension pairs) are ranked by a **quality metric** / “cognostic” (computed diagnostic), then shown with 1D/2D/nD techniques.
- **Feature extraction** (= dimensionality reduction): *transform* existing features into a new, lower-dimensional space (project onto new axes). The new axes are usually *not*

interpretable, and extraction *loses information*.

- **Hybrid:** select fewer meaningful dimensions first (needs domain knowledge), then apply dimensionality reduction on them.

7.1 Curse of Dimensionality (Giraud)

As the dimension p grows, high-dimensional spaces become **vast** and data points get **isolated in their immensity**. Three intertwined effects:

- **Distance concentration:** the notion of “nearest neighbor” vanishes. For n points sampled uniformly in $[0, 1]^p$, as p increases the *minimal* distance between two points increases and *all* pairwise distances become almost equal – every point is roughly equally far from every other. The mean square distance grows linearly with p , $\mathbb{E}[\|X^{(i)} - X^{(j)}\|^2] = p/6$, while the *relative* spread (scaled deviation) shrinks like $p^{-1/2}$. Any method based on **local averaging** (e.g. k -Nearest-Neighbors, kernel smoothing) therefore fails.
- **Sparsity:** to keep observations within distance 1 of any query point, the required number of samples n grows *more than exponentially* with p . The lower bound is $n \gtrsim (\frac{p}{2\pi e})^{p/2} \sqrt{p\pi}$; e.g. for $p = 100$ already $n \approx 4 \cdot 10^{39}$, for $p = 200$ more than the estimated number of particles in the observable universe. Real data is hopelessly sparse in high- p space.
- **Vanishing volume:** the volume $V_p(r)$ of a p -ball goes to zero more than exponentially fast in p ; for $p \approx 20$ the unit ball is already almost 0. Geometric intuitions from 2D/3D are misleading.
- **Cumulating fluctuations:** accumulation of many small fluctuations across directions produces a large global fluctuation; rare events accumulate and may no longer be rare; computations/optimizations become intensive.

“More features is always better”

The “blessing” of measuring thousands of variables is offset by the curse: *separating signal from noise is in general almost impossible* in high-dimensional data. More dimensions $\neq$ more usable information.

7.2 Feature Selection vs. Feature Extraction (Ghojogh)

Both reduce dimensionality: a mapping $\mathbf{x} \in \mathbb{R}^d \mapsto \mathbf{y} \in \mathbb{R}^p$ with $p \leq d$.

- **Feature selection:** $p \leq d$ and the selected set $\{\mathbf{y}^j\} \subseteq \{\mathbf{x}^j\}$ is an *inclusive subset* of the original features. Selection criterion improves/maintains accuracy or simplifies the model. There are 2^d possible subsets, so heuristics are used. Two evaluation philosophies:
 - **Filter methods:** rank/score features *independently* of any learning model, then threshold. Based on **relevance** (predictive power w.r.t. target) and **redundancy**. Measures: correlation criteria (Pearson), mutual information / information gain, χ^2 statistics, Markov blanket, consistency-based filter, FCBF (symmetrical uncertainty), Interact, mRMR (max relevance, min redundancy). Fast, model-agnostic.
 - **Wrapper methods:** *integrate the model* into the subset search – train and test the classifier on each candidate subset and use its performance as fitness. Search is NP-hard, so heuristic: sequential forward/backward selection (SFS/SBS, also floating SFFS/SFBS) or metaheuristics (genetic algorithms, particle swarm). More accurate but expensive; risk of overfitting to the model.
 - **Embedded methods:** selection happens *during* model training (e.g. LASSO / regularization, tree importances). Between filter and wrapper in cost.
- **Feature extraction:** the new set $\{\mathbf{y}^j\}$ is *not* a subset but a *different space* (projection to new axes); often $p \ll d$. Relies on the **manifold hypothesis**: data lie on a lower-dimensional

sub-manifold. Split into *linear* (PCA) vs *non-linear* (kernel PCA, MDS, t-SNE, UMAP, ISOMAP) and *unsupervised* (uses only variation, e.g. PCA) vs *supervised* (uses labels, e.g. LDA).

7.3 Visual Methods for Multivariate Tabular Data

These work for a handful to a few dozen dimensions, then break down.

Parallel Coordinates

Core idea: each dimension is a *vertical axis*, placed side by side; each item is a *polyline* crossing all axes at its values.

Key properties: reading correlation between *adjacent* axes — **positive correlation** $\Rightarrow$ roughly *parallel* line segments; **negative correlation** $\Rightarrow$ *crossing* (X-shaped) segments. Good for spotting clusters and trends across many attributes.

Hyperparameters / choices: *axis ordering* (only adjacent pairs are directly comparable, so order matters a lot), axis scaling, opacity.

Scalability: suffers heavily from **overplotting** with many items; $\sim$ dozens of axes at most.

When to use: moderate #dimensions, want to compare per-item profiles and adjacent-pair correlations.

Pitfalls: non-adjacent correlations are invisible; a poor axis order hides structure; clutter with large n .

Star / Radar Plots (Glyphs)

Core idea: a per-item **glyph**: axes radiate from a center, the value on each axis is a spoke length, endpoints connected into a star polygon.

Key properties: compact shape signature per item; good for comparing a *few* items across a *few* attributes; can be tiled as small multiples.

Hyperparameters: axis order (shape depends strongly on it), normalization.

Scalability: few items, few dimensions; does not scale to many of either.

When to use: profile comparison of individual records/categories.

Pitfalls: area is misleading (not a meaningful quantity); shape depends on dimension order; hard to read with many overlapping glyphs.

Scatterplot Matrix (SPLOM)

Core idea: a grid of all pairwise 2D scatterplots (m dimensions $\rightarrow m \times m$ matrix, $\binom{m}{2}$ distinct pairs); diagonal often shows the 1D distribution.

Key properties: every pairwise dependency/correlation is visible; color by class label to inspect separability.

Scalability: number of cells grows as $O(m^2)$ – breaks down around $\sim$ dozens of dimensions; overplotting per cell with large n .

When to use: explore all pairwise relations; basis for ranking cells via **scagnostics** / **class consistency**.

Pitfalls: quadratic blow-up; only *pairwise* (no higher-order) structure; too small cells become unreadable.

Other tabular techniques

- **Table lens / heatmaps:** the $n \times m$ table itself shown as a color matrix (cell value $\rightarrow$ color). Scales to many items and dimensions; rows and columns can be reordered (seriation) to reveal block structure. Used e.g. for gene expression and demographic indicators.
- **Glyphs** (general): map data attributes to visual channels of a small mark (position, color, shape, size, tilt). Star/radar plots are one family.

- **Star Coordinates** (Kandogan): items as points; each dimension is a unit vector from the origin, the point is $\sum_i u_i(d_{ji} - \min_i)$. Interactive: scaling an axis = feature emphasis, rotating = change correlation, switching off = feature selection. *Strongly depends on dimension order.*

7.4 Ranking Views: Scagnostics & Class Consistency

These are **quality metrics** for feature *selection*: rank scatterplots (dimension pairs) so the analyst sees the most “interesting” ones first.

Scagnostics (Wilkinson et al., 2005)

Core idea: *scatterplot diagnostics* – a set of measures computed *per scatterplot* to characterize its shape, used to **rank** the cells of a SPLOM.

The nine measures:

- **Outlying** – presence of outliers; via minimum spanning tree (MST): $c_{\text{outlying}} = \text{length}(T_{\text{outliers}})/\text{length}(T)$, outliers = degree-1 vertices with edge weight $> 1.5\text{IQR}$.
- **Skewed** – skewness of the edge-length distribution.
- **Clumpy** – points form tight clumps/clusters.
- **Sparse** – points spread thinly / low density.
- **Striated** – parallel strips/bands of points.
- **Convex** – how convex the cloud is: $c_{\text{convex}} = \text{area}(A)/\text{area}(H)$, A = alpha hull, H = convex hull.
- **Skinny** – elongated, thin shape.
- **Stringy** – thin, string-like (path-like structure in the MST).
- **Monotonic** – monotone trend (rank correlation).

When to use: too many dimension pairs to inspect by hand; surface plots with outliers / clusters / trends automatically.

Pitfalls: purely shape-based (ignores labels); a measure can be fooled by sampling/binning; not all “interesting” plots score high.

Class Consistency (Sips et al., 2009)

Core idea: for *labeled* data, rank projections by how well classes are *visually separated*.

How: distance-consistency (DSC) – the *ratio of data points whose closest class centroid is their own class’s centroid*. High DSC = classes map to visually separable regions; low DSC = classes overlap. (Slide example: DSC= 90 vs DSC= 49 scatterplots.)

When to use: pick the dimension pair / projection that best separates known classes (e.g. for visual inspection of classification).

Pitfalls: needs labels; centroid-based, so non-convex / multi-modal classes can be misjudged.

Related: **rank-by-feature** (Seo & Shneiderman) ranks by 1D criteria (normality, uniformity/entropy, #outliers, #unique values) or 2D criteria (correlation coefficient, regression error, density); **parallel coordinates in Hough space** (Tatu et al.) – good dimension pairs have fewer, well-defined clusters in Hough space.

7.5 Dimensionality Reduction Fact-Sheets

PCA – Principal Component Analysis

Core idea: linear projection that **maximizes variance** along each new axis, equivalently **minimizes the reconstruction (reprojection) error** (squared distance between original and projected points).

How: operate on *normalized* data (centered to mean, scaled to unit variance). Principal components = **eigenvectors of the covariance matrix** S (solving $S\mathbf{u} = \lambda\mathbf{u}$). The *first* PC is the eigenvector with the *largest eigenvalue* (direction of max variance); PCs are **mutually orthogonal** and form a new coordinate system.

Properties: deterministic; global, linear; preserves large pairwise distances; “score plot” = projection onto first 2 PCs (proximity indicates similarity); **cannot capture non-linear structure** (fails on the swiss roll). Often only a small fraction of variance is kept (MNIST PCA $< 20\%$).

Hyperparameters: number of components k to keep (scree plot, cumulative explained-variance threshold).

Complexity: eigendecomposition of $d \times d$ covariance ($O(d^3)$, or SVD); fast and scalable.

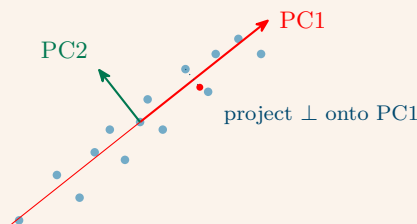
When to use: linear correlation structure, need reproducibility, interpretable variance, pre-processing before t-SNE/UMAP.

Pitfalls: only linear; sensitive to scaling (*must* standardize); 2D projection can be a poor approximation if variance is spread over many PCs.

Sketching a 1D PCA projection

A classic exam task: given a 2D point cloud elongated along a diagonal, draw the **first principal component** (the long axis of the cloud, through the mean, direction of maximum variance) and the second PC orthogonal to it. To get the **1D projection**, drop a *perpendicular* from each point onto PC1 and read off its position along that line.

Sketch: Ellipse-shaped cloud; arrow PC1 along the long axis through the centroid; short orthogonal arrow PC2; for each point a thin line dropped at a right angle onto PC1; the feet of those perpendiculars are the 1D PCA scores.



MDS – Multidimensional Scaling

Core idea: find a low-dimensional embedding that **best preserves the pairwise distances** $\delta_{i,j}$ of the high-dimensional data.

Cost (stress):
$$\min_{\mathbf{x}_1, \dots, \mathbf{x}_n} \sum_{i < j} (\|\mathbf{x}_i - \mathbf{x}_j\| - \delta_{i,j})^2.$$

Variants: **classical / metric** MDS preserves actual distance values (classical MDS on Euclidean distances $\approx$ PCA); **non-metric** MDS preserves only the *rank order* of distances. Distances can be any metric (e.g. cosine similarity for documents). **ISOMAP** = MDS on **geodesic** (manifold) distances instead of Euclidean – a non-linear variant that unrolls manifolds like the swiss roll.

Properties: preserves global distance structure; like PCA tends to keep *dissimilar* points far apart.

Hyperparameters: target dimension, metric, metric vs non-metric.

Complexity: works on the $n \times n$ distance matrix $\Rightarrow O(n^2)$ memory/time, heavier for large n .
When to use: you have a meaningful (dis)similarity matrix and care about preserving overall distances.
Pitfalls: struggles to keep *very similar* points close (crowding); $O(n^2)$ does not scale to huge n .

t-SNE – t-Distributed Stochastic Neighbor Embedding

Core idea: preserve **local neighborhoods**, not distances. Model high-dimensional similarities as conditional probabilities (Gaussian), $p_{j|i} \propto \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$, and low-dimensional similarities with a heavy-tailed **Student-t** distribution $q_{j|i}$; minimize the **Kullback–Leibler divergence** $C = \sum_i \text{KL}(P_i \| Q_i)$ by gradient descent.

Key properties: the Student-t tail solves the **crowding problem** (room to spread out medium distances). Keeps similar points close; **stochastic** / **non-deterministic** (different runs differ).

Hyperparameters: **perplexity** (effective #neighbors defining the Gaussian variance; balances local vs global, ≈ 5 –50), learning rate (ϵ), iterations; can use Barnes–Hut approximation.

Complexity: heavier than PCA/UMAP; iterative optimization ($O(n^2)$ naive, $O(n \log n)$ with Barnes–Hut).

When to use: reveal local cluster structure (e.g. MNIST digit clusters).

Pitfalls (heavily examined): *cluster sizes are NOT meaningful; inter-cluster distances are NOT meaningful*; results depend on perplexity and random seed; can show clusters in random data at low perplexity; preserves no global geometry.

UMAP – Uniform Manifold Approximation and Projection

Core idea: **manifold** / **graph-based**. Build a weighted k -nearest-neighbor graph (a fuzzy topological / simplicial representation of the manifold), then optimize a low-dimensional layout with the closest equivalent fuzzy topological structure.

Key properties: like t-SNE preserves **local** neighborhoods but tends to preserve **more global structure**; **faster** and scales better; still **stochastic**.

Hyperparameters: **n_neighbors** (size of local neighborhood from which the manifold is learned – larger = more global view), **min_dist** (how tightly points may pack in the embedding – smaller = tighter clumps).

Complexity: approximate kNN $\Rightarrow$ scales to large n , faster than t-SNE.

When to use: large datasets needing local *and* some global structure (single-cell, embeddings).

Pitfalls: absolute distances/densities still not fully reliable; hyperparameter-dependent shapes; non-deterministic.

Distortions & the structure–interpretability trade-off

Any DR projects feature space $\mathcal{D}$ to visual space $\mathcal{M}$ and introduces **distortions**: **missing neighbors** (close in feature space but far in the plot) and **false neighbors** (far in feature space but close in the plot). Trade-off (Liu et al., 2017): **manifold learning** (t-SNE, UMAP) captures the most intrinsic structure but has *non-interpretable* axes; **axis-aligned projection** (feature selection) has fully interpretable axes but least structure; **linear projection** (PCA) sits in between. Take-away: *feature extraction leads to information loss*.

7.5.1 Comparison Table: PCA vs MDS vs t-SNE vs UMAP

Method	Linear?	Determ.?	Preserves	Main hyperparameters	Scalability
PCA	yes	yes	global variance / large distances; linear subspace	#components k	high ($O(d^3)$)
MDS	no*	yes (classical)	global pairwise distances (stress)	target dim; metric; (non)metric	low ($O(n^2)$)
t-SNE	no	no	local neighborhoods (cluster sizes & inter-cluster dist. <i>not</i> meaningful)	perplexity , learning rate ϵ , iterations	medium
UMAP	no	no	local + more global structure (kNN graph / manifold)	n_neighbors , min_dist	high (fast)

*Classical/metric MDS on Euclidean distances is linear and coincides with PCA; non-metric MDS and ISOMAP are non-linear.

MCQ-style: properties & hyperparameters

- Which method is deterministic (same result every run)? → PCA (and classical MDS). t-SNE and UMAP are **stochastic**.
- Whose cluster sizes and inter-cluster distances are meaningless? → **t-SNE** (and largely t-SNE-style layouts).
- “Perplexity” is a hyperparameter of ... → **t-SNE** (number of neighbors defining the Gaussian variance, balances local/global).
- “n_neighbors” and “min_dist” belong to ... → **UMAP**.
- Which is the only linear method? → **PCA** (and classical MDS).
- PCA finds the directions of ... → **maximum variance** = eigenvectors of the covariance matrix; first PC = largest eigenvalue.
- Which best preserves global pairwise distances? → MDS / PCA; which best preserves local neighborhoods? → t-SNE / UMAP.

Over-interpreting a 2D projection

Do not read meaning into t-SNE/UMAP *distances*, cluster *sizes*, or empty gaps; do not call a t-SNE result reproducible; do not forget to *standardize* before PCA; remember that a low total explained variance means the 2D PCA scatterplot is a weak summary. State that projection axes are *not interpretable* for feature-extraction methods.

8 Scalable Information Visualization & Big Data

Big data (Heer & Kandel) has three dimensions: data can be **tall** (billions/trillions of rows / items), **wide** (hundreds/thousands of attributes / variables), and **diverse** (many sources, varied types). A pragmatic visualization definition: *big data* = when our traditional visual encodings break. When each data record is mapped to one mark, naive encodings (scatterplots, parallel coordinates) become unreadable already at 10 000–100 000 items.

The exam-critical distinction: two kinds of scalability

There are **two** separate scalability challenges. Naming both correctly is the most common exam point.

- **Perceptual scalability** — can the user *read* it? The display has fewer pixels than data points; too many marks cause **overplotting**, occlusion, clutter, low readability. Limited by *human perception* (precision of the eye/mind) and *display resolution*, not by the database. *Goal: show a larger amount of data in a readable way* [Richer et al. 2022].
- **Interactive / computational scalability** — is it *fast* enough? Too much data to query/process at interactive rates: inefficient database queries, inefficient data processing, rendering-performance limits, interaction latency.

Rule of thumb (latency): the **100 ms** interactive response-time limit. Beyond it, exploration stalls. A Google study found that adding just **200 ms** latency to search measurably reduced the number of searches — and the effect persisted for weeks (Heer & Kandel).

8.1 Factors Affecting (Visual) Scalability

List: factors affecting scalability (Erick & Karr 2012)

Factor	Affects
Human perception (precision of eye & mind)	perceptual
Monitor resolution (physical size, #pixels)	perceptual
Visual metaphors (choice of idiom & channels)	perceptual
Interactivity (panning, zooming, brushing)	both
Data structures & algorithms (graph layout, data cubes)	interactive
Computational infrastructure (CPU, GPU, network)	interactive

The first three sit on the perceptual side; interactivity bridges; the last two are the interactive/computational side. Also relevant: #rows, #dimensions, memory.

List: sources of latency (Heer & Kandel)

The interaction round-trip has three latency stages, each with its own fix:

1. **Data query** — searching too much data (e.g. `month=1` in 4 M rows). *Fix:* pre-computed aggregates / data cubes; plus data transfer (network) and disk I/O.
2. **Data processing** — aggregation/computation too slow. *Fix:* parallelization (map-reduce style), prefetching.
3. **Visualization / rendering** — drawing too slow for interaction/animation (e.g. SVG/D3 does not scale with #items). *Fix:* bitmap/GPU rendering (WebGL, WebGPU) instead of SVG.

8.2 Overplotting: Simple Solutions vs. Preprocessing

Each data record → one visual item. With big data this gives **occlusion**, **clutter**, poor **readability**, and a **waste of resources**.

Simple solutions that need NO data preprocessing

These touch only the *rendering*, not the data:

- **Alpha blending** (opacity/transparency) — overlapping marks darken, revealing density. (Cannot show outliers + global patterns at once.)
- **Smaller marks** / reduced point size.
- **Jittering** — small random offsets to separate coincident marks.

- **Draw order** (z-order) of marks.
- **Open vs filled** marks (outline-only marks occlude less).

Contrast — solutions that DO need preprocessing: binning / aggregation, sampling, kernel density estimation. The slides’ “3 simple solutions” are **alpha blending**, **filtering**, **random sampling**; the warning: *these cannot show global patterns and outliers at the same time.*

Do not mix up the two solution classes

Alpha blending, jittering, smaller/open marks, draw-order = *no preprocessing*. Binning, aggregation, density estimation, sampling = *preprocessing*. If asked for “simple solutions to overplotting that need no preprocessing”, listing binning loses the point.

8.3 Aggregation & Density: Scalable Encodings

Core idea: *visualize **models** or **aggregates** of attributes instead of individual data points* — individual items are combined. The perceptual scalability of a display should be limited by the chosen *resolution of the data*, not by the number of records (Heer & Kandel).

Binning / 2D histograms

Idea: split the data range into regular **intervals (bins)**; encode the **frequency** (count per bin). Quantitative/ordinal → regular intervals; nominal/categorical → one bin per category.

Scalable because we control the *number of bins* independently of #records; same visual encoding as a bar chart (length/position).

Hyperparameter: bin size / number of bins.

Pitfall: sensitive to bin size — can cause **aliasing** effects (e.g. 9 vs 10 bins, 10×10 vs 30×30 bins look very different).

Hexagonal binning (hexbin)

2D binning where bins are **hexagons** instead of squares.

Why hexagons:

- **Nearest-neighbour symmetry** — all 6 neighbours are at the *same distance* (a square grid mixes edge- and corner-neighbours, introducing visual bias).
- Hexagons are the *maximum-side regular polygon that tessellates the plane* → most “circular”, efficient density approximation.

Frequency encoded by *colour or size*. **Hyperparameter:** bin size / number of bins. Used in hexbin scatterplots, hexbin SPLOMs (Carr et al. 1987), and geographic hexbin maps (e.g. Walmart / farmers’ markets in the U.S.).

Kernel Density Estimation (KDE)

Non-parametric estimate of the probability density function of a random variable by weighting distances of observations $x_1, \dots, x_n$ from a query point x :

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i).$$

Hyperparameters:

- **Bandwidth h** (kernel width): the larger h , the *smoother* the curve. *Too small* → **under-smoothing** (noisy, spiky); *too large* → **over-smoothing** (a single blob; range and mode become misleading).
- **Kernel K** : non-negative, integrates to 1, mean 0 — usually a *Gaussian* (a rectangular/box

kernel also works).

Used in: violin plots (density + embedded box plot, good for **multimodal** distributions), density scatterplots (2D KDE), geographic heatmaps. Munzner’s *continuous scatterplots* use the same idea: a derived *overplot-density* value coloured per pixel instead of discrete dots.

Sampling (random / stratified)

Show a *representative subset* with a bounded number of points instead of all records.

Random sampling: simple, uniform; risks dropping rare outliers.

Stratified sampling: sample within groups/strata so each is represented — better coverage of minority categories.

Trade-off: fewer marks → less overplotting and faster rendering, but *outliers and fine structure can be lost*. Can be combined with aggregation (aggregate views of well-chosen samples approximate the full data).

8.4 Data Cubes / Multi-Dimensional Tables (OLAP)

For *interactive* scalability, the killer technique is to **pre-aggregate whatever possible** so that interactive queries hit small precomputed tables, not the raw rows.

Data cubes / multi-dimensional tables

From flat to cube: a flat table (1 key → list, 2 keys → matrix/contingency table) generalizes to an n -dimensional **data cube**. Each *cell stores a precomputed aggregate* (**sum**, mean, count, min/max).

OLAP operations:

- **Roll-up** (projection): aggregate away a dimension (e.g. project onto (Time, Products, *)).
- **Drill-down:** the inverse — go to a finer level of detail.
- **Slice:** fix one dimension to a single value (e.g. Texas).
- **Dice:** select a sub-range on several dimensions.

Why it enables interactive scalability: brushing/linking, dynamic queries, panning/zooming become lookups + small re-aggregations over the cube → within the 100 ms budget.

Cost (key pitfall): size of the cube = $\prod_i b_i$ (product of bin counts per dimension) — **exponential in the number of dimensions**. So pre-compute *wisely* (only frequent / low-order projections; supports simple, not arbitrary compound and/or queries).

imMens / Nanocubes idea

imMens [Liu, Jiang & Heer 2013] and **Nanocubes** [Lins et al. 2013]: precompute **binned aggregates** (multivariate data tiles) so that **real-time brushing & linking** over hundreds of millions of records stays interactive (e.g. Brightkite ~4M check-ins: brush “January” in a month histogram → map + day + hour views update instantly). Brushing/linking = filter + new aggregation; panning/zooming = change of bin resolution / #samples.

8.5 Reducing Processing & Rendering Latency

Techniques (Heer & Kandel; slides)

- **Parallel aggregation** / data-parallel processing (map-reduce style, GPU/clusters). Requirements: data must be **separable** (can be split and re-composed, e.g. local histograms merged into a global one) and **streamable** (processed independently). Tools: Dask, RAPIDS, PySpark.
- **Prefetching & predictive caching** — fetch data tiles the user is likely to request next

(e.g. adjacent zoom levels).

- **Progressive / online aggregation & sampling** — show an approximate result immediately and refine it, so the user is never blocked waiting for the exact answer.
- **GPU / bitmap rendering** (WebGL, WebGPU) instead of SVG (D3), which does not scale with $\#items$.

8.6 Reduce Strategy (Munzner ch. 13)

Filter vs Aggregate vs Embed

Reduce is one of Munzner’s strategies for managing complexity. It applies to both *items* and *attributes* (= “elements”).

- **Filter** — simply *eliminate* elements (item filtering / attribute filtering), often via **dynamic queries** (tightly-coupled encoding + interaction; e.g. FilmFinder, scented widgets). *Risk*: “out of sight, out of mind” — users forget filtered-out data.
- **Aggregate** — replace a group of elements by a single *derived* element that stands in for them (count, sum, mean, min/max). Cognitively safer (conveys info about the whole group) but *cannot convey everything omitted*; the challenge is summarizing without hiding signal (cf. Anscombe’s quartet). Examples: histograms, continuous scatterplots, box plots.
- **Embed** — show focus and context together in one view (focus+context), so detail and overview coexist without removing data.

8.7 Interactive Exploration: Shneiderman’s Mantra

Overview first, zoom and filter, details on demand [Shneiderman 1996] maps directly onto scalable techniques:

- **Overview** via aggregate visualizations (binned/hexbin maps, heatmaps, choropleths).
- **Zoom** via **hierarchical aggregation / semantic zooming** (drill-down): objects change shape/detail and new objects appear — not just resizing. Aggregate visualizations are very well suited to this.
- **Filter** via **brushing & linking** / dynamic queries.
- **Details on demand**: tooltips that report *summary statistics of the aggregate / bin* (e.g. a hexbin tooltip: $\#users$, $\#posts$, $\#locations$ in that bin).

Worked connection: 10M traffic accidents

Task: explore ~10 M traffic-accident records (location, time, severity) interactively. Both scalability challenges appear — name and solve each:

Perceptual scalability 10 M point marks on a map $\rightarrow$ total overplotting. *Solution:* **aggregate**. Bin *spatially* (regions/states, or a hexbin grid) and *temporally* (by year). Encode counts by colour $\rightarrow$ **choropleth** or **hexbin density map** (space) and a **line chart** of accidents per year or a **histogram** per region (time).

Interactive scalability Re-querying 10 M rows on every brush exceeds 100 ms. *Solution:* build a **data cube** over (region $\times$ year $\times$ severity) with precomputed counts; brushing/linking then becomes cube lookups + small re-aggregations. Cost $\prod_i b_i$ stays small because only a few coarse dimensions are binned.

Linked views A **choropleth/hexbin map** linked to a **line chart / histogram**: brush a year on the line chart $\rightarrow$ map re-aggregates; brush a region $\rightarrow$ time chart updates. Add details-on-demand tooltips reporting per-bin summary stats. If still slow: parallel aggregation + prefetching + progressive refinement.

One-liner answer: *bin spatially & temporally for perceptual scalability; data cube for interactive scalability; linked choropleth/hexbin map + line chart.*

How this is asked

- “Name and distinguish the two scalability challenges” → perceptual vs interactive/computational (see top keybox).
- “List factors affecting scalability” / “list sources of latency” → the two list-keyboxes above; map each to perceptual/interactive.
- “Give simple overplotting solutions needing no preprocessing” → alpha, smaller/open marks, jitter, draw-order (NOT binning/sampling).
- “Why hexagons?” → uniform neighbour distance + best regular tessellation.
- “KDE bandwidth too large/small?” → over-/under-smoothing.
- “Why are data cubes expensive?” → size $\prod_i b_i$, exponential in #dimensions.
- “Munzner reduce” → filter vs aggregate vs embed.

9 Network and Tree Visualization

This is a high-priority exam topic: graph-drawing algorithms appear under “Visualization Algorithms”, and node-link ↔ matrix and treemap ↔ node-link tree appear under “Mapping data items” / “Alternatives”. The aesthetic-criteria list and the force-directed / layered / morphing / foresighted layouts are recurring question targets.

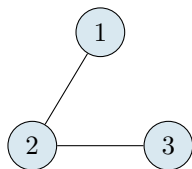
9.1 Graph basics

Definitions

A **graph** $G = (V, E)$ is an abstract structure for modelling *relational* information: V is a set of **nodes** (objects), E a set of **edges** (relations connecting nodes).

- **Directed** vs. **undirected**: edges may have a direction. In a directed graph the **in-degree** of a node is the number of incoming edges, the **out-degree** the number of outgoing edges. The **degree** is the number of incident edges.
- **Weighted**: edges (or nodes) can carry **values** (edge weights), possibly of different types.
- Graphs may contain **cycles**; special families: planar, bipartite, hierarchical, clustered graphs, and **hypergraphs** (an edge connects more than two nodes).
- **Tree** = special case of a graph: *no cycles*, one special **root** node, every non-root node has exactly one parent. Variants: free, binary, rooted, ordered trees.
- **Network** = graph + attributes on nodes/edges. A **multivariate network** attaches many attributes per element (primary data: directly measured; secondary data: derived/computed).

Data structures and the two visual representations. Internally a graph is stored as an **adjacency matrix** ($|V| \times |V|$, entry = 1 / weight if edge present) or an **adjacency list**. For *display* there are two fundamental mappings of the same data:



Node-link diagram

	1	2	3
1	0	1	0
2	1	0	1
3	0	1	0

Adjacency matrix

9.2 Node-link vs. adjacency matrix (mapping & alternatives)

The two are the classic **alternative representations** of the *same* relational data, with complementary strengths.

	Node–link	Adjacency matrix
Edge → visual mark	line/curve between two node marks	filled cell (i, j) (colour/size → weight)
Path following / topology	good – follow lines, see connectivity, clusters by spatial proximity	hard – must hop row→column mentally; multi-hop paths very tedious
Density / scalability	degrades on dense graphs (hairball, many edge crossings, occlusion)	good on dense graphs: <i>no edge crossings</i> , no overlap, every edge visible
Clusters / cliques	visible if layout separates them	a clique = solid block on the diagonal; easy to spot with good row/column ordering
Space	compact for <i>sparse</i> graphs	always $ V ^2$ cells

One-line rule of thumb

Sparse graph + need to *follow paths / read topology* ⇒ **node–link**. *Dense* graph + want to spot *clusters/cliques* and avoid crossings ⇒ **matrix**. Hybrid **NodeTrix** (Henry et al. 2007) draws dense communities as small matrices and the sparse links between them as node–link edges.

9.3 Aesthetic criteria for graph drawing (the “List” topic)

A good layout should be *easy to read, understand and remember*. Layout algorithms try to satisfy a set of **aesthetic criteria**; these are often *contradictory* and their optimisation is mostly *NP-hard*, so most graph-drawing algorithms are **heuristics**.

Aesthetic criteria – memorise the list

- Minimise **edge crossings** ↓
- Minimise **bends** of edges ↓ (max / uniform / total bends)
- Maximise **symmetry** ↑
- **Edge length** ↓: short, and *uniform* edge lengths
- Even **node distribution**; avoid **node overlap**
- Minimise drawing **area** ↓
- Maximise **angular resolution** (“resolution” ↑): angles between edges at a node should be large
- Show **clusters** / community structure

Criteria conflict

The criteria *trade off*. Classic exam example: a K_4 drawn as a *triangle with a centre node* has *zero crossings* but no symmetry; drawn as a *square with both diagonals* it is fully *symmetric* but has one crossing. You cannot maximise everything at once – say so explicitly.

9.4 Layout algorithm: Force-directed / spring embedder

Force-directed (spring embedder, Eades 1984)

Idea. Model the graph as a *physical system*: edges act as **springs** (attractive force pulling adjacent nodes together, $\sim$ spring toward a rest length), and *all* pairs of nodes exert a **repulsive** force (like charges / “gravitation between nodes”). Iterate: compute net force on each node, move it a little, repeat until the system reaches a **stable, low-energy configuration** (energy minimisation).

Properties. Produces *symmetric, organic* layouts with *uniform edge lengths* – exactly the two

criteria Eades targeted. Works well on small/medium sparse graphs; reveals clusters. Generalises to 3D.

Hyperparameters. Spring constant / ideal (rest) edge length, repulsion strength, step size or **temperature** (cooling schedule, as in Fruchterman–Reingold), number of **iterations**, initial (often random) placement.

Complexity. Naive: $O(n^2)$ repulsion per iteration (all node pairs) $\times$ iterations. **Barnes–Hut** approximation groups distant nodes into a quad/oct-tree and treats each group as one pseudo-node $\Rightarrow O(n \log n)$ per iteration.

Pitfalls. *Non-deterministic* (depends on random start); gets stuck in *local minima*; classic spring embedder has *high runtime* and can *break down on very large graphs*. Improvements: Kamada–Kawai 1989 (local minimal energies), Fruchterman–Reingold 1991, Davidson & Harel 1996 (simulated annealing).

9.5 Layout algorithm: Layered / Sugiyama (hierarchies, flow)

Layered graph drawing (Sugiyama–Tagawa–Toda 1981)

Idea. For *directed* graphs / DAGs, hierarchies, and flow: arrange nodes in horizontal *layers* (rows) so all edges point downward. Classic pipeline:

1. **Cycle removal** – extract an acyclic subgraph that contains all nodes (temporarily reverse a few edges).
2. **Layer assignment** – give each node an integer layer number; nodes arranged top-down in rows. (Long edges get *dummy nodes* on intermediate layers.)
3. **Crossing minimisation** – choose the *order* within each row to reduce edge crossings, typically layer-by-layer (e.g. **barycenter** heuristic: put a node at the average position of its neighbours in the adjacent layer).
4. **Coordinate assignment** – assign final x/y to straighten edges and balance spacing.

Properties. Emphasises *direction* / *hierarchy* / *flow*; good for DAGs, dependency and process diagrams. Variants: *parallel layers* (top-down rows) and *radial layers* (concentric rings).

Hyperparameters. Layer-assignment policy (longest-path, network-simplex), crossing-minimisation heuristic and #sweeps, node/layer spacing.

Complexity. Even 2-layer crossing minimisation is *NP-hard* $\Rightarrow$ heuristics (barycenter, median).

Pitfalls. Cycles must be handled first; dummy nodes inflate the graph; poor for undirected / non-hierarchical graphs.

9.6 Dynamic graphs and the mental map

Networks increasingly *change over time* (biochemical pathways gain new reactions, social contacts appear/disappear). Visualisations of such dynamic graphs must **preserve the “Mental Map”** (Misue, Eades, Lai, Sugiyama 1995): the user should *recognise old structures again* across time steps, so nodes should not jump around between frames.

Graph morphing

Idea. Visualise the transition between two layouts with *smooth animation*: if a node changes position, compute an *interpolated animation path* between old and new position. Can be applied on top of *any* 2D/3D layout algorithm (e.g. GraphAEL uses force-based layouts).

+ Looks good, eases tracking. – Nodes *still change position*; newly added or deleted nodes may not be correctly identified. This is an **online** approach: lay out each step, then animate between consecutive steps.

Foresighted layout (Diehl, Görg, Kerren 2001)

Idea. If the *whole sequence* of graphs is known (or can be pre-computed) – an **offline** setting – build a **supergraph** (union of all time steps) and compute *one* layout from it. Position nodes at the start so that they *do not change their positions later* across the whole sequence.

+ *Best mental-map preservation; independent* of the chosen layout algorithm. – The sequence is *often unknown* in advance; partly poor aesthetics (e.g. gaps at the beginning where future-only nodes are reserved).

Online vs. offline; animation vs. juxtaposition

Online: layout per step as data arrives (morphing animates between them). **Offline:** full sequence known $\Rightarrow$ foresighted layout for stable positions. Time can be shown by **animation** (one view, moving) or by **small multiples / timeline** (many static views side by side, e.g. *matrix cubes*: stack adjacency matrices into a space-time cube).

9.7 Tree layouts

Tree drawing families

- **Node-link trees: rectilinear** (classic top-down/left-right hierarchy with root on top) or **radial** (root in centre, depth $\rightarrow$ radius).
- **Indented / outline** layout: nested indentation (like a file explorer); depth $\rightarrow$ horizontal indent.
- **Space-filling treemap:** *recursive subdivision* of a rectangle. Hierarchy is encoded by **containment** (child rectangles nested inside the parent); a *leaf's* value is mapped to its **area**. No wasted whitespace. Algorithms: **slice-and-dice** (alternate cut direction per level – simple but produces thin slivers) vs. **squarified** (keep cells close to square – better aspect ratio, easier area comparison).

Mapping data items: treemap $\leftrightarrow$ node-link tree

Treemap and **rectilinear node-link tree** are two encodings of the *same hierarchy*. Node-link shows *structure/depth* explicitly but wastes space and scales poorly; treemap is *space-filling* and great for comparing *leaf values by area* but makes the *topology/depth* hard to read. Containment (treemap) vs. connection (node-link) is the underlying encoding difference.

9.8 Biological networks (Kerren)

Modern **systems biology** treats biological processes as integrated systems of many interacting components; high-throughput methods (since the human genome project) produce huge data, and **networks are the key concept** to structure and combine it.

Types of biological networks (a hierarchy)

- **Molecular graphs** – atoms/bonds of a single molecule.
- **Metabolic networks / pathways** – nodes are substances, edges are reactions, each single reaction catalysed by an **enzyme**; a **pathway** is a sequence of reactions converting one substance into another (databases: *KEGG*; tools: PathViewer, BioPath, VANTED).
- **Protein-protein interaction (PPI) networks** – nodes are proteins, edges are interactions (e.g. *signal transduction*); fundamental for many functions and diseases (tool: *Cytoscape*).
- **Gene regulatory networks** – regulatory relations between genes.

Why it matters: diseases interpreted in a network context (infection = a foreign network)

operating in ours; genetic defect = incorrect connectivity); applications in drug design and metabolic engineering.

Challenges & integrative bioinformatics

Challenges: *scale* (huge networks, scalability/navigation); *integrating multiple heterogeneous data types* onto network elements (multivariate primary/secondary attributes); respecting *standard pathway-map conventions* (e.g. KEGG layouts biologists expect).

Integrative bioinformatics combines network topology with the attached multivariate data. State-of-the-art strategies (cf. the multivariate-network slides):

- **Integrated approaches** – replace nodes by embedded diagrams (e.g. bar charts in pathway nodes).
- **Multiple coordinated views** – network view linked with parallel coordinates, heatmaps, etc. (brushing-and-linking; e.g. Caleydo).
- **Semantic substrates** – node position fixed by attributes in non-overlapping regions; edge slider controls clutter.
- **Attribute-driven topology** / hybrid (JauntyNets) – user balances whether *topology* or *attributes* drive the force layout.

9.9 Exam boxes

MCQ: node-link vs. adjacency matrix

“For a large, very dense network where the analyst wants to find tightly connected communities, which representation is preferable, and why?”

Answer: the **adjacency matrix**. On dense graphs node-link becomes a hairball with many crossings/occlusion, whereas a matrix has *no edge crossings*, shows *every* edge, and – with good row/column ordering – makes cliques appear as solid diagonal blocks. Trade-off: *path following* across multiple hops is much harder in a matrix, so for sparse graphs / topology tracing, prefer **node-link**.

MCQ: force-directed vs. layered

Which layout for an undirected social network where symmetry and uniform edge length matter? $\Rightarrow$ **force-directed**. Which for a directed DAG / dependency or process flow? $\Rightarrow$ **layered** / **Sugiyama**. Distractor traps: force-directed is *non-deterministic* and prone to *local minima* (true); Sugiyama’s crossing minimisation is *NP-hard* hence heuristic (true); force-directed is naively $O(n^2)$ per iteration, reducible to $O(n \log n)$ via **Barnes–Hut** (true).

MCQ: morphing vs. foresighted layout

Both aim to **preserve the mental map** of a *dynamic* graph. **Morphing** = *online*, animate (interpolate) between per-step layouts; nodes still move, can be applied to any layout, but added/removed nodes are hard to follow. **Foresighted layout** = *offline*, needs the full sequence, computes a *supergraph* so node positions *stay stable* across all time steps; best mental-map preservation but sequence often unknown and aesthetics can suffer (gaps). Common wrong answer: claiming foresighted works online – it does *not*; it requires the sequence in advance.

10 Complex Data: Sets, Curves, and Ensembles

Classic multidimensional data has records with numerical and categorical attributes. **Complex data** adds richer attribute kinds: **sets**, **curves** (function graphs / time series), and **surfaces**. This section covers three such cases that recur in the exam: set-typed data (how to visualise it, with

UpSet as the modern go-to answer), families of function graphs (Konyha et al.), and simulation ensembles (Matkovic et al.). All three are united by one idea: an attribute is no longer a single scalar but a structured object, and the same visual-analysis toolkit (**coordinated multiple views** + **brushing and linking**) is used to explore them.

10.1 Set-Typed Data

Definition

Set-typed data: a collection of **elements**, each of which has a membership in one or more of several *possibly overlapping sets*. Equivalently, each element carries a **set-typed attribute** whose value is a subset of the universe of sets (a **set family** / set system over the elements). Sets may relate by **containment, exclusion, and intersection**.

Example (movies). Universe of sets = genres {Romance, Drama, Comedy, ...}. Each movie is an element; its set-typed attribute is its genre set, e.g. *Cinema Paradiso* = {Romance, Drama, Comedy}, *Touch* = {Romance}. Other real-world cases: club memberships, product features, employee skill sets, search-engine result overlaps, patient symptoms.

Key notions (Sets-STAR, Alsallakh et al.).

- **Set membership degree** of an element = number of sets it belongs to (its **degree**).
- **Exclusive membership:** an element belongs to exactly one set / one specific intersection.
- **Intersection / overlap:** the set of elements shared by a group of sets. With n sets there are up to 2^n possible membership combinations, which is why naive approaches do not scale.
- Data representations: cardinalities only, multi-valued attribute (adjacency-list style), membership lists (edge-list style), or Boolean attributes (adjacency-matrix style).

Typical tasks (what a technique must support): find elements of a set, find sets of an element, filter by membership degree; find the number of sets; analyse inclusion / exclusion / intersection relations; identify the sets in a given intersection and the largest / smallest intersections; compare set and intersection cardinalities and similarities.

10.1.1 Techniques (Sets-STAR categories)

Euler / Venn diagrams

Idea: sets = labelled closed curves (circles, ellipses, polygons); set relations = curve overlaps. A **Venn** diagram shows *all* 2^n possible overlaps; an **Euler** diagram shows only the overlaps that actually occur (it can be **well-matched**: regions reflect the true relations).

Reads: membership pops out preattentively via enclosure.

Trade-offs: *does not scale beyond ~ 3 sets*; for ≥ 4 sets, well-matched + well-formed (simple, smooth, area-proportional) curves are often impossible, and *some intersections become undrawable*. Area-judgement bias hurts area-proportional variants.

Over-plotting / point-based (overlays)

Idea: elements live in an existing layout (scatterplot, map, graph) and set membership is overlaid with **colour/glyphs** (coloured dots, pie-glyphs, hatching) or with **region** overlays (**Bubble Sets**, texture splatting) or **line** overlays (**LineSets**, **Kelp Diagrams**).

Use when: elements have a meaningful spatial reference and set membership is secondary. Region overlays handle ~ 4 -20 sets.

Trade-offs: limited number of sets/elements; layout artifacts (spurious overlaps, crossings); overlays cannot move the elements.

Matrix-based

Idea: a **set-membership matrix** – rows = elements, columns = sets (cells mark membership); or sets $\times$ sets cells holding a similarity / overlap measure (heatmap). Reordering rows/columns reveals clusters of similar membership. Examples: ConSet, PixelLayers, frequency grids.

Trade-offs: fairly scalable in both elements and sets, clutter-free (no edge crossings), but limited in which set *relations* they express, and patterns are *sensitive to ordering*.

Node-link / graph-based

Idea: model element–set membership as a **bipartite graph** (edges = memberships). Examples: Jigsaw lists, anchored maps, PivotPaths; Radial Sets / Circos for set–set links.

Trade-offs: elements are visible as individual objects and clusters of similar membership show up, but *limited scalability due to edge crossings*; element–set diagrams show no set relations directly.

Aggregation-based

Idea: when elements are many, follow “overview first”: aggregate elements into a single visual element encoding a *frequency*. Bar charts of set/intersection sizes, **Set’o’gram** (Freiler et al.), Mosaic / Double-Decker plots, Parallel Sets, **Radial Sets** (Alsallakh et al.).

Trade-offs: *highly scalable in number of elements*; some show how attributes correlate with membership; but usually do not show single elements as objects and are limited in set relations.

UpSet – the modern go-to technique

Lex, Gehlenborg, Strobel, Vuilleumot & Pfister, *UpSet: Visualization of Intersecting Sets*, IEEE TVCG 2014. **This is the standard answer to “how do you visualise set-typed data with many sets?”**

UpSet replaces the overlapping regions of an Euler/Venn diagram with a **matrix of intersections** plus **bar charts**:

- **Set-size bars** – one bar per set giving each set’s cardinality.
- **Intersection matrix** – columns are the (non-empty) intersections; each column is a vertical strip of dots over the set rows. A **filled dot** = “in this set”, an **empty dot** = “not in this set”; dots in the same column are joined by a line. A column with one filled dot = “only in that set”; all dots filled = “in all sets”.
- **Intersection-size bars** – a bar chart giving the cardinality of each intersection, usually *sorted* (e.g. by size) so the dominant combinations are obvious.

Because intersections are laid out as a sortable list rather than packed into the plane, UpSet *scales to many sets and many elements* and makes the hard set tasks (which sets are in an intersection? which intersection is largest?) directly readable. Related slide techniques: OnSet (binary set matrices with set operations), Radial Sets, Set’o’gram.

Common deductions

Do not draw a Venn diagram for ≥ 4 sets and claim it “works” – some intersections are undrawable and area judgements are biased. Do not show set sizes as a **pie chart**: set categories are not mutually exclusive and do not sum to a whole. For “many sets” always reach for **UpSet** (or aggregation/matrix methods), not Euler/Venn.

Sketch: An UpSet plot has three coupled parts. *Top-left:* a small bar chart of total set sizes (sets A, B, C). *Bottom-left:* the set rows A, B, C as labels. *Centre/bottom:* a dot matrix – one column per intersection, each column a vertical run of filled (member) / empty (non-member) dots, filled dots in a column joined by a line. *Top:* a horizontal bar above each column whose length is that

intersection’s cardinality, columns sorted by size. Reading a column’s dots tells you exactly which sets define the intersection; the bar tells you how big it is.

10.1.2 Comparison of set-visualisation categories

Category	Scales in #sets	Scales in #elements	Shows inter-sections?	Order matters?
Euler / Venn	~3 (poor)	tens	yes, but some undrawable for ≥ 4	layout-driven
Overlays (point/region/line)	4-20	tens–hundreds	partially (regions show overlap)	element positions fixed by base view
Node-link (bi-partite)	tens	hundreds	set–set only (Circos/Radial); not in element–set	sensitive (crossings)
Matrix-based	up to ~100	up to ~100s	limited relations	yes (reordering reveals patterns)
Aggregation (Set'o'gram, Radial Sets, Double-Decker)	tens~50	large (hundreds+)	limited / on demand	yes
UpSet	many	many	yes, explicitly + sizes	sortable (by size/degree)

10.2 Families of Curves / Function Graphs

Definition

A **function graph** $f(\mathbf{x}, t)$ is a dependent variable plotted over an independent variable (e.g. time): one 2D curve per data record. A **family of function graphs** is the set of all such curves for one dependent variable across all records – many curves drawn in the same view. Source: Konyha, Matkovic, Gračanin, Jelović & Hauser, *Interactive Visual Analysis of Families of Function Graphs*, IEEE TVCG 2006.

Data model. Records have m **independent variables** $\mathbf{x} = [x_1, \dots, x_m]$ and n **dependent variables**. Dependents are either **regular** (a single scalar per record) or **function-graph** variables (a value for each time t , i.e. a curve). A curve can be treated either as an **atomic item** or as an **attribute / dimension** of a record (cf. the slides’ table with a **Pressure(t)** / **Velocity(t)** column). Examples: traffic-sensor occupancy/volume over a day (the Minneapolis data set: ~94,752 graphs, 144 values each), animal paths, ergonomics, fuel-injection rate curves.

Core problem: over-plotting. Thousands of curves (e.g. 45,000) overlap into a dense band, so individual curves and regions of maxima/minima are hidden.

Interactive visual analysis of curve families

Setup: the **curve view** (a family of function graphs) embedded in **multiple coordinated linked views** alongside histograms, scatterplots, parallel coordinates, and a map of the independent variables. Brushing in any view highlights the same items (**focus**) in all others; context is shown desaturated (**focus+context**).

Density rendering: draw curves semi-transparently so pixels through which more graphs pass are rendered with higher intensity – a **banded / density representation** that reveals the shape of a dense family.

Brushing in function space:

- **Line brush** – a line segment in the curve view selects all curves that *cross* it. Intuitive, easy to define, would be far more complex as an SQL query.
- **Rectangular / polyline brush** – selects curves passing through a region (or, with edge constraints, curves with a local max/min inside it).

Brushing in derived attribute / parameter spaces: aggregate each curve into scalar **curve features** (max, min, slope, value-at- x , mean) and brush those in scatterplots/histograms; link back to the curves and to the independent variables.

Composite brushing: combine brushes with logical **AND, OR, SUB** to iteratively drill down or broaden the focus.

Analysis procedures. Black-box reconstruction: understand how dependent curves change as independent variables vary (overview → zoom/filter → details). **Inverse analysis:** brush a desired curve shape, then find which independent-variable combinations produce it. **Multidimensional relations:** brush in one curve family and study the linked curves of another. A **colour gradient** along a brush links brushed items across views.

10.3 Simulation Ensembles

Definition

A **simulation ensemble** is a set of many **simulation runs** (ensemble **members**), each produced by the same model with different **control parameters / inputs**. Formally a member is a pair $(\mathbf{x}^i, \mathbf{y}^i)$ where $\mathbf{x} \in \mathbb{R}^n$ are control parameters and $\mathbf{y} = S(\mathbf{x})$ the outputs; the ensemble is $E = \{(\mathbf{x}^i, \mathbf{y}^i)\}$. Members are **multi-dimensional and often time-dependent** (scalars, curves, surfaces, fields). Source: Matkovic, Gračanin & Hauser, *Visual Analytics for Simulation Ensembles*, WSC 2018.

Why ensembles. Two questions drive simulation: (1) what is the output for a given parametrisation (easy – run it), and (2) which parametrisation gives a desired output (hard – inverting the model). Realistic models cannot be inverted analytically, so we simulate *many* parametrisations and explore the ensemble. Sampling of the parameter space uses full-factorial or low-discrepancy (Sobol) designs (e.g. the common-rail Diesel injector: $5 \times 5 \times 5 \times 5 \times 7 = 4375$ runs).

Goals / tasks. Parameter sensitivity – relate outputs to control parameters and find which parameters matter; **model reconstruction** – find parameters for a desired output; **comparison** of outputs from different parameter regions; identifying **representative** and **outlier** members.

Visual analytics for ensembles

Coordinated multiple views (CMV): link *parameter space* (left, scatter-plots/histograms/parallel coordinates of control parameters), the *curve/time views* (centre, e.g. needle-lift curves), and other *derived-output* views (right). A consistent left-to-right parameter→output layout helps build a mental model.

Brushing & linking across members: brush a region of parameter space or a desired output range; corresponding members highlight everywhere. Use the curve **line brush** (cross-the-line selection) on the time-dependent outputs.

Aggregation: reduce complex (non-scalar) outputs to **derived scalar features** (min/max of a curve, etc.); support **on-the-fly** aggregation during analysis so the session need not be restarted.

Interactive ensemble steering: start with a coarse sampling, then *initiate new simulation*

runs from within the analysis where more detail is needed (the ensemble grows as you explore). Three loops: inner analysis loop (linking/brushing in CMV), ensemble-refinement loop (new parametrisations / rows), model-refinement loop (new model parameters / columns). **Hybrid steering** adds an automatic regression-model optimisation to guide the interactive search. **Reproducibility:** structure the brushing space (snap-to-grid, percentile grids), overlay statistics, and quantify findings so qualitative insights (“the curves rise steeply”) become reproducible numbers.

Sketch: Ensemble CMV: left column = parameter-space scatterplots/histograms ($dT_v, dT_p, P_{low}, \dots$) with a brushed region; centre = a curve view of all members’ output curves, brushed members in orange, context in grey; right = scatterplots of derived scalar outputs; a data table below lists the brushed runs. Brushing any view re-highlights the same members everywhere.

10.4 Exam Angle

Set-typed data: strategy / list questions

“What is set-typed data and how would you visualise it?” (1) Define: elements with membership in multiple, possibly overlapping sets (set-typed attribute). (2) List techniques by category: Euler/Venn, overlays (point/region/line), matrix-based, node-link, aggregation. (3) State the key trade-off: Euler/Venn fails beyond ~ 3 sets (undrawable intersections, area bias). (4) **Recommend UpSet** for many sets: intersection matrix + sorted intersection-size bars + set-size bars; scales, shows intersections explicitly. Mention the pie-chart pitfall and that pattern visibility in matrices depends on ordering.

Reading an UpSet plot – visualisation literacy

You may be shown an UpSet plot and asked to read it. Steps: (a) the left bars give each *set’s* total size; (b) pick a column in the matrix – the *filled* dots name the sets the elements belong to, *empty* dots the sets they do not; (c) the bar above that column gives the *intersection’s size*. So “only B ” = one filled dot on B ; “ $A \cap B$ but not C ” = filled A, B , empty C ; “ $A \cap B \cap C$ ” = all filled. The tallest intersection bar is the dominant combination. Equivalent question for curves: “a line brush selects which curves?” – all curves that *cross the line*.

Curves & ensembles

Expect short-answer questions: define a **family of function graphs** (many curves of one dependent variable; problem = over-plotting; remedies = density/banded rendering, line/composite brushing, brushing on derived curve features in linked views). Define a **simulation ensemble** (many runs with varied parameters; members multi-dimensional/time-dependent; goals = parameter sensitivity, model reconstruction, representative/outlier members; technique = CMV linking parameter space $\leftrightarrow$ spatial/curve views, aggregation, brushing across members, plus interactive ensemble steering).

11 Visualization for Machine Learning (VIS4ML) & Explainable AI

VIS4ML = *visualization to support machine-learning problems*. It is the dual of **ML4VIS** (machine learning used to build or steer visualizations, e.g. clustering and KDE used to summarize a scatterplot). **Mnemonic:** *VIS4ML* – the visualization is the tool, the ML model is the *what* (the data); *ML4VIS* – ML is the tool inside the visualization pipeline.

VIS4ML in one sentence

Use interactive visualization across the whole ML pipeline – to understand the **data**, to **build/debug** models, to **evaluate** them, and to **interpret** them and build **trust** – because models (and their training data) are high-dimensional black boxes.

11.1 Why VIS4ML: visualization in the ML pipeline

After Sacha et al. (VIS4ML Ontology, 2019) the **what** is always *models*; visualization supports the phases of the (visual) data-science process:

- **Train** – training-data inspection; training-progress inspection (watch the learning over epochs); interactive labeling.
- **Evaluate** – model-quality analysis; **explainability**; **interpretability**.
- **Use** – what-if analysis.

Model evaluation answers three questions (a common exam framing):

- (1) **IF** the model learned correctly (e.g. confusion matrix).
- (2) **WHAT** the model learned (e.g. projecting learned features).
- (3) **WHY** the model did *not* learn correctly (e.g. similarity-preserving scatterplots of neuron activations).

VIS4ML vs. ML4VIS

VIS4ML: visualization helps the ML practitioner (debug/understand the model). **ML4VIS**: ML helps the visualization (e.g. projection, clustering, KDE-thresholding to find clusters in screen space).

11.2 Confusion matrix – “IF the model learned correctly”

A **confusion matrix** cross-tabulates ground truth against predictions for a classifier. Convention used in the course: **rows = true (actual) label**, **columns = predicted (estimated) label**; each cell holds the frequency (count) of items.

- **Diagonal** cells = correct predictions (true class = predicted class).
- **Off-diagonal** cells = errors; cell (i, j) tells you how often true class i was predicted as j , i.e. *which classes get confused*.
- **Normalization**: dividing each row by its row sum turns counts into per-class recall ratios, so classes with different support are comparable (a stacked-bar / “confusion star/gear” variant shows the accumulated ratio).

true\pred.	c_1	c_2	c_3
c_1	200	5	4
c_2	2	156	3
c_3	2	3	211

Sketch: A square grid; bold diagonal = hits; bright off-diagonal cell = a systematic confusion (e.g. on MNIST many true 9’s land in the “4” column).

Visualization-literacy: reading a confusion matrix

This is a recurring exam item – you are given a matrix and asked to read it.

- “*Boxing is never confused with handwaving.*” $\Rightarrow$ the cell (true = boxing, pred = handwaving) is **0**. “Never confused” = zero off-diagonal cell. (Both directions $\rightarrow$ check both cells.)
- “*Which class is hardest to classify?*” $\Rightarrow$ the class whose **diagonal value is lowest** relative

to its row total, i.e. the row whose mass is **most spread** across off-diagonal columns (lowest recall). With normalization, the smallest diagonal percentage.

- “Which class is best classified?” $\Rightarrow$ highest normalized diagonal entry.

11.3 Feature vectors from neuron activations (Rauber et al.)

Rauber et al. (TVCG 2017, mandatory reading) treat the **activations of a layer** as a high-dimensional feature vector and project them to 2D.

- For a multilayer perceptron, neuron j in layer l has weighted input $z_j^{(l)} = b_j^{(l)} + \sum_k w_{j,k}^{(l)} a_k^{(l-1)}$ and activation $a_j^{(l)}$ (sigmoid, ReLU = $\max(0, z)$, or softmax). The layer activation $\mathbf{a}^{(l)} = (a_1^{(l)}, \dots, a_{N^{(l)}}^{(l)})$ is a **learned representation** of the input.
- These vectors are projected with a **similarity-preserving** technique (**t-SNE**, UMAP, or hierarchical SNE) into a scatterplot. Color = ground-truth class. The *axes are not interpretable, but the instances are*.

What the projection of activations shows

- **Single-color clusters** (one class = one well-separated blob) $\Rightarrow$ identical ground-truth classes produce similar activations $\Rightarrow$ *solid model performance*.
- Class **separation increases with depth and with training**: deeper layers / later epochs separate classes more cleanly than the raw input.
- **Overlapping / mixed-color** regions $\Rightarrow$ classes the network cannot tell apart – exactly the confusions seen off-diagonal in the confusion matrix.
- **Visual outliers** (a point of one color inside another cluster) flag problematic / misclassified cases (e.g. a “3” that activates like a “5”).

Caveat (FMNIST): a similarity-preserving scatterplot *alone* may not suffice to explain patterns/outliers – inspect representative instances too.

11.4 Interpretability taxonomy (XAI)

Three orthogonal axes

Scope

Local = explain the decision for a *single instance*; **Global** = explain the *whole model* behavior. (A globally complex model can still have locally explainable decision boundaries.)

When

Intrinsic = the model is inherently interpretable (read the model itself); **Post-hoc** = explain after training with a separate method.

Coupling

Model-specific = exploits a known, inherently interpretable structure; **Model-agnostic** = treats the model as a *black box* (inner structure too complex or unknown), probing only via inputs and outputs.

Model-specific / intrinsic examples: **linear regression** ($y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + \epsilon$; read the weights β with confidence intervals – but note weights mix with feature scales, so use $effect = \beta_j x_i^j$ plots, not raw weights, to judge contribution), **decision trees** (splitting condition, Gini impurity, samples-per-class per node), and CNN feature visualizations of spatial activations.

Exam “List”: name the model-agnostic methods

1. **Local perturbations** (probe one instance by editing its features).
2. **LIME** – Local Interpretable Model-agnostic Explanations.
3. **SHAP** / Shapley-value feature attribution.
4. **Partial Dependence Plots (PDP)** and **ICE** curves.
5. **Counterfactual explanations**.
6. **Saliency** / **attention maps** (gradient/attribution based).

All are post-hoc and need only input–output access (no internal weights).

11.5 Model-agnostic methods

Local perturbations / LIME (Ribeiro et al. 2016)

Idea: explain one prediction by *perturbing the input around that instance* and observing how the black-box output changes; fit a *simple, interpretable local surrogate* (e.g. sparse linear model) to the perturbed samples to approximate the decision boundary nearby.

Scope / coupling: **local**, **model-agnostic**, post-hoc.

Properties: works on **interpretable features** (need not be quantitative – e.g. image **super-pixels** masked in/out, or a 3D object’s texture/hue altered). Reveals what a single decision relies on (e.g. a model keying on *texture rather than shape*).

Hyperparameters: perturbation/sampling neighborhood (kernel width), number of samples, surrogate complexity (# features kept).

Pitfalls: explanation depends on the (arbitrary) neighborhood definition; only locally faithful; unstable – nearby instances can get different explanations.

SHAP / Shapley feature attribution

Idea: from cooperative game theory – distribute the prediction fairly among features as **Shapley values**, the average marginal contribution of a feature over all orderings of feature coalitions.

Scope / coupling: **local** attribution (aggregates to global), **model-agnostic**.

Properties: additive, consistent attribution per feature; positive vs. negative contributions to a specific prediction.

Pitfalls: exact computation is exponential in # features (approximated in practice); like PDP it can be misled by correlated features.

Partial Dependence Plot (PDP) – Krause et al. 2016

Idea: the *marginal effect* of one (or two) feature(s) on the prediction, *averaged over all other features’ values in the data*. Systematically set the chosen feature to each level (try all recorded levels for categorical features), re-predict for every training row, average the responses, and plot level vs. average response.

Scope / coupling: **global**, **model-agnostic**.

Reading: a curve (quantitative feature) or bar chart (categorical), e.g. predicted bike rentals rise with temperature, fall with humidity/wind.

Hyperparameters: which feature(s); grid resolution.

Pitfall: **assumes feature independence** – averaging over the marginal can create unrealistic feature combinations and hide heterogeneous effects. **ICE (Individual Conditional Expectation)** curves are the *per-instance* variant: one line per data point instead of the average, exposing interactions the averaged PDP masks.

Counterfactual explanations

Idea: the **minimal set of changes to the feature values** that *flips* the prediction to a *different output* – the smallest perturbation that crosses the decision boundary.

Scope / coupling: **local**, **model-agnostic**.

Properties: *not unique* – many valid counterfactuals exist (several different small changes can each flip the label), so often a set is sought. A good counterfactual should be **plausible** / **realistic** (lie on the data manifold) and **sparse** / **close** (change few features, stay near the original). What-if tools (ViCE, Gomez et al. 2020) show the change as a glyph against the feature’s density distribution.

Pitfalls: can be unrealistic if plausibility is not enforced; multiplicity makes “the” explanation ambiguous.

Counterfactual true/false MCQ

- “A counterfactual is the *minimal* set of changes that flips the prediction.” $\Rightarrow$ **TRUE**.
- “A counterfactual is the *maximum* change to feature values.” $\Rightarrow$ **FALSE** (it is minimal, not maximal).
- “There is exactly *one* counterfactual per instance.” $\Rightarrow$ **FALSE** (counterfactuals are not unique).
- “Counterfactuals should be *plausible*.” $\Rightarrow$ **TRUE**.

Saliency / attention maps (brief): post-hoc, typically **local** maps that highlight which input regions (pixels, tokens) most influenced a single prediction – a heatmap overlay answering “where did the model look?”.

11.6 Surrogate models

A **surrogate model** is a simple, interpretable model (linear model, shallow tree) trained to mimic a black-box model’s input–output behavior. Used *locally* (LIME’s local surrogate around one instance) or *globally*, surrogates make black-box models interpretable by substituting a transparent approximation that humans can read.

Common confusions

- Confusion-matrix axes: *rows = truth*, *columns = prediction*. Reversing them flips your reading of every off-diagonal cell.
- PDP/SHAP *assume feature independence*; do not claim PDP gives per-instance behavior – that is ICE.
- Counterfactual = *minimal* change and *not unique*; “maximum” or “only one” are wrong.
- Model-agnostic $\neq$ model-specific: agnostic = black-box, input/output only; specific = exploits a known interpretable structure.

12 Evaluation and Validation

How do you know your visualization actually **amplifies cognition**? You cannot rely on intuition: the design space is huge and most designs are ineffective. Evaluation answers “how well does my visualization work?”; the exam phrases this as “how will you validate your proposed solution?”. The central framework is Munzner’s **nested model** (Munzner, TVCG 2009; book Chapter 4), which tells you, level by level, what can go wrong (**threats**) and which method addresses each threat.

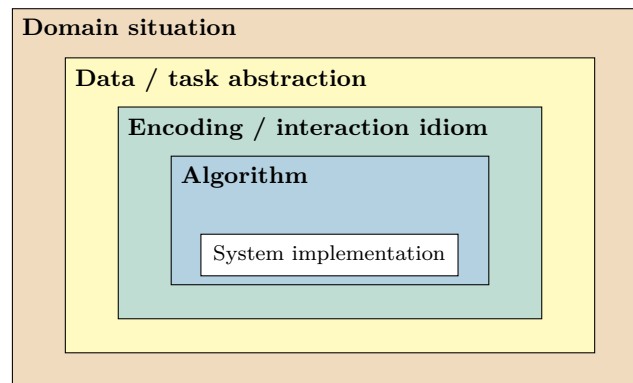
12.1 Munzner's Nested Model: Four Levels

Vis design is split into four **cascading, nested** levels. The output of an **upstream** (outer) level is the input to the **downstream** (inner) one. A block is the outcome of design at a level. The big consequence of nesting: a **poor choice at an upstream level cascades to all downstream levels**, so a perfect idiom and algorithm cannot save a wrong abstraction.

The four levels (outer → inner)

1. **Domain situation** (*why* / *who*): target users, their domain, questions, data. Blocks are *identified*.
2. **Data / task abstraction** (*what* & *why*): map domain-specific data + questions into domain-independent data types and abstract tasks. Tasks are identified; data is *designed*.
3. **Visual encoding / interaction idiom** (*how*): how to draw it (visual encoding) and how the user changes it (interaction). Blocks are *designed*.
4. **Algorithm**: efficient automatic procedure that instantiates the idiom. Blocks are *designed*.

Sketch: four nested rectangles, outer to inner: Domain $\supset$ Data/Task abstraction $\supset$ Encoding/Interaction idiom $\supset$ Algorithm, with “System implementation” as the innermost white box.



12.1.1 Threats and Validation per Level

Each level has its own **threat** (the way that level can fail) and its own **validation method**. Crucially, validations are either **immediate** (can be done at that level, before downstream levels exist) or **downstream** (require the inner levels to be implemented first, so they cannot be done until the system runs). Immediate validation only gives *partial* evidence; downstream validation is necessary for a full demonstration.

Level	Threat	Validation method	Timing
Domain situation	Wrong problem (you misunderstood their needs)	Observe / interview target users (contextual inquiry, field study) Measure / observe adoption rates of deployed tool	Immediate Downstream
Data / task abstraction	Wrong abstraction (you show them the wrong thing)	Test on real target users doing their own tasks ; collect anecdotal evidence of utility Field study of the deployed system in real workflow	Downstream Downstream
Encoding / interaction idiom	Wrong idiom (the way you show it does not work)	Justify design vs. alternatives using perceptual / cognitive principles; heuristic / expert review	Immediate

Level	Threat	Validation method	Timing
Algorithm	Wrong / slow algorithm (your code is too slow or incorrect)	Qualitative / quantitative result-image analysis	Downstream
		Lab study: controlled experiment measuring human time and errors for a task	Downstream
		Analyze computational complexity	Immediate
		Measure wall-clock system time and memory ; compare to benchmarks	Downstream

Mismatch between claim and method

A weak project validates at the wrong level. You *cannot* validate a new *encoding idiom* with wall-clock algorithm timings, and you *cannot* address a mischaracterized *task* with a lab study whose task is dictated by the experimenter. **Choose the validation method according to your contribution claim.** Informal usability studies are done *upstream* of a real study (to fix usability bugs first) but are *not* themselves a validation method.

12.2 Types of Evaluation

Two broad styles, often combined (mixed-methods):

- **Quantitative:** numeric measures, statistical testing (lab/online experiments, eye tracking metrics, algorithm benchmarks).
- **Qualitative:** observations, interviews, open-ended feedback, coded transcripts.

Method catalogue (by number of people involved, large → few)

- **Controlled experiment / lab study:** comparative, in a controlled setting; measures performance. Strongest for comparing encoding / interaction choices.
- **Crowdsourcing study** (Mechanical Turk, Prolific): web workers, micro-payments. + diverse/large population (ecological validity, large design-space coverage); – no control over conditions. Needs **attention checks** / instructional manipulation checks.
- **Eye tracking study:** records fixations, saccades, scanpaths; metrics per **area of interest (AOI)**.
- **Insight-based study:** no predefined tasks; users explore until “done”, think-aloud, insights **coded** and quantified, treated as dependent variables (North, 2006).
- **Expert / heuristic evaluation:** “discount” method, ~5 experts find >75% of problems (Nielsen); checks against vis heuristics (Shneiderman’s mantra, perception, color blindness). ICE-T questionnaire operationalizes value = Time + Insight + Essence + Confidence. Cognitive walkthrough = expert steps through specific tasks.
- **Usability study:** uncovers usability problems; user need not be in the target domain. Done before a validating study, not instead of it.
- **Field study:** qualitative, on-site, users work on their own problems in their normal environment. “Gold standard but time consuming”. Validates domain (existing practice) or abstraction (changed behavior after deployment).
- **Case study / qualitative result inspection:** no experiment; reader inspects result images / a walkthrough (comparative or isolated). A reasonable middle ground.
- **Qualitative coding:** iterative characterization of transcripts / video / field notes. **Open coding:** label each phenomenon (concept), group into categories, build a **code book**. Quality assurance via multiple coders and **inter-rater reliability**.

- **Longitudinal study:** observes use over an extended period (e.g. adoption, field deployment over time).
- **Algorithmic / benchmark evaluation:** complexity analysis plus wall-clock time and memory on standard benchmarks.

12.3 Controlled Experiment Design

A controlled experiment isolates the effect of one (or few) factors on measured performance, ensuring **internal validity**.

Anatomy of a controlled experiment

- **Hypothesis:** logical, precise, testable, justifiable (state the expected direction).
- **Independent variables (factors):** what you deliberately vary (e.g. visualization method, task, data size, population). **Levels** = number of variations per factor = test conditions.
- **Dependent variables:** the measured outcome (user performance).
- **Control variables:** held constant (e.g. display size, data characteristics).
- **Random variables:** randomly varied across conditions (e.g. data variations).
- **Confounding variables:** unmeasured variable that unintentionally correlates with an IV; must be avoided to keep internal validity (example: crosses are harder to click than circles, so a difference attributed to “encoding” is really about clickability).

List: typical dependent variables

- **Task completion time** (speed; ideally only the actual solving time, shown after instructions).
- **Accuracy / error rate** – including **false positives** and **false negatives**.
- Number of interaction steps / actions (logged clicks, mouse moves).
- **Subjective preference / satisfaction** (Likert scale, questionnaires e.g. System Usability Scale).
- **Confidence, memorability/recall, and insight count** (for exploratory tasks).
- Eye-tracking measures: first-fixation time, number of fixations in an AOI, saccade amplitude, scanpath transitions.

12.3.1 Experimental Design and Statistics

- **Within-subjects** (repeated measures): every participant does every condition. Watch for **learning effect** → **counterbalance** condition order (e.g. **Latin square**).
- **Between-subjects:** each group does only one variation of a factor (no learning effect, but needs more participants, more variance).
- **Mixed design:** combination of the two.
- **Power analysis** decides the number of participants given α (Type I error), β (Type II error / power = $1 - \beta$) and effect size. Increase power by more participants, fewer conditions, larger effect size, lower variance (homogeneous population, removing outliers, more control variables).
- **Statistical significance:** check distribution (e.g. Shapiro test); if normal use a (matched) *t*-test / ANOVA with post-hoc (Tukey), otherwise a non-parametric test (Wilcoxon signed-rank).

Useless user studies

Foregone conclusions, fishing for results, confounding factors, experimenter bias, lack of external validity (not generalizable), lack of practical significance, lack of power. Antidote: define research

questions / success criteria *before* you start; then the validation method is usually obvious.

12.4 Worked Example: Encoding Classes in Static Scatterplots

Template answer to “design a user study to compare visual encodings of classes (sets) in a static scatterplot”. The task is to mark group membership of points; candidate channels are **color**, **shape**, **connection** (Gestalt), and **redundant combinations**.

Full study template

Hypotheses (directional, justified by channel rankings / perception):

- H1: **color** > **shape** – color (a position-free identity channel) is more effective than shape per the channel ranking.
- H2: **connection** > color – connection lines (Gestalt grouping) beat color for grouping tasks.
- H3: **redundant** > single – redundant encoding (e.g. color + shape) beats either single channel (less channel interference).

Independent variable: encoding $\in$ {color, shape, connection, combinations} (one factor, ≥ 4 levels).

Dependent variables: **accuracy** – with **false positives** (items wrongly marked) and **false negatives** (items missed) – and **task completion time**; optionally subjective preference.

Task: “click / select all items of a given class.”

Confounds to avoid: marks differ in clickability (crosses are harder to click than circles) $\rightarrow$ equalize target size / hit area; also label readability, visual appeal.

Control variables: number of classes, class balance, number of points, point distribution, chart size, display.

Design: within-subjects with counterbalanced order; power analysis for N ; significance test on accuracy and time.

12.5 Effectiveness: Why We Visualize Defines the DV

The choice of task and dependent variable is not arbitrary: it follows from **why** people use the visualization. Munzner’s task abstraction distinguishes low-level **actions** – **identify**, **compare**, **summarize** – from the data **targets** the user is after – **trends**, **outliers**, **features**, distributions, correlation, similarity, topology / paths. Effectiveness means: faster to interpret, supports more distinctions, fewer errors. So a “find the outlier” goal becomes an *identify* task with accuracy + time as DVs; a “which is bigger” goal becomes a *compare* task. Match the measured DV to the real low-level action, or the study answers the wrong question.

“How would you validate your project / design a user study?”

A safe full-marks structure:

1. **Locate the contribution in the nested model** and pick the matching method (encoding/interaction claim $\rightarrow$ lab study; algorithm claim $\rightarrow$ benchmarks; problem claim $\rightarrow$ field study; adoption $\rightarrow$ longitudinal).
2. **State a precise, justified hypothesis** and the high-level goal (exploration / confirmation / presentation).
3. **Name IVs and their levels** (encoding/technique, data size, task), **DVs** (completion time, accuracy with FP/FN, preference, insight count), **control** and **confounding** variables.
4. **Choose design** (within- vs between-subjects, counterbalancing), **participants** via power analysis, and the **statistical test**.
5. Add **qualitative data** (think-aloud, semi-structured interview, coding) to explain outliers

and put numbers in context.

6. Discuss **threats / limitations**: representativeness of tasks, layout, number and background of users; whether “time + error tell the whole story”.

Remember the two big takeaways: *validation method follows the contribution claim*, and *upstream errors cascade downstream*.

13 Exam Question-Type Playbook

This section gives, for each of the nine exam question types foreshadowed by the professor, (a) what it asks, (b) a step-by-step answer strategy, (c) the rubric / point deductions to avoid, and (d) a short worked mini-example. The detailed theory lives in the earlier sections; here we drill the *format of the answer*.

13.1 Type 1 – Visualization Literacy (read a chart)

What it asks. Given a concrete visualization (boxplot, bar/stacked/ grouped bar, histogram, pie, line chart, scatterplot, parallel coordinates, star/radar, SPLOM, heatmap, confusion matrix, choropleth, node-link, adjacency matrix, treemap, mosaic, UpSet, arc diagram, 3D scatter), report observations in free form or judge true/false statements.

Answer strategy

1. **Identify the chart** and what is mapped to each channel (x, y, colour, size, area, position).
2. **Read the encoding back to data**: a tall bar = large value, dark heatmap cell = large value, wide mosaic tile = large marginal.
3. For **distributions** (box/histogram): state median, spread/IQR, skew, outliers, multimodality.
4. For **relations** (scatter/SPLOM/PCC): state correlation sign, clusters, outliers, trends.
5. For **confusion matrices**: diagonal = correct; off-diagonal cell (i, j) = class i predicted as j . “Hardest to classify” = row with smallest diagonal / most off-diagonal mass.
6. Answer each T/F by tracing the exact cell / mark, not intuition.

Pitfall / point deduction

Confusing rows vs. columns in a confusion matrix (true class is usually the row). Reading area as length (pie/treemap/mosaic encode area, which is perceptually weak). Asserting correlation from a single outlier. Forgetting that a log axis compresses large values.

Mini-example. Confusion matrix: cell (boxing, handwaving) = 0 $\Rightarrow$ “boxing is never confused with handwaving” is **True**. The class with the smallest diagonal entry relative to its row sum is “jogging” $\Rightarrow$ “jogging is hardest to classify” is **True**.

13.2 Type 2 – Visualization Algorithms (describe / sketch / MCQ)

What it asks. Describe, sketch, or answer multiple-choice on the properties and hyperparameters of a named algorithm: recursive subdivision (mosaic, treemap), just-noticeable-difference, scagnostics, class consistency, PCA, MDS, t-SNE, UMAP, hexbin, KDE, data cubes, parallel aggregation, feature vectors, local perturbations, partial dependence, counterfactuals, force-directed / layered / morphing / foresighted graph layout.

Answer strategy

1. State the **core idea** in one sentence (input $\rightarrow$ output).
2. Name the **key hyperparameters** and the effect of each.

3. State **properties**: linear vs. nonlinear, deterministic vs. stochastic, preserves global vs. local structure, complexity.
4. If asked to sketch, draw axes, a few labelled points, and the result (e.g. the projection line for PCA, the binned grid for hexbin).

Pitfall / point deduction

Claiming t-SNE/UMAP distances or cluster *sizes* are meaningful (they are not). Saying PCA is nonlinear. Forgetting t-SNE/UMAP are stochastic (need a seed) and perplexity / *n_neighbors* controls local-vs-global balance. See the Algorithm Cheat-Sheet (Section 14) for the full table.

Mini-example. “Sketch the 1D PCA projection of a 2D point cloud.” $\Rightarrow$ draw the cloud, the first principal axis along the direction of maximum variance, and the points projected (dropped perpendicularly) onto it.

13.3 Type 3 – Visualization Strategy (data + task $\rightarrow$ design)

What it asks. Given a dataset and an analysis task, propose and justify data processing + visual design + interaction design, and defend it against alternatives. Data may be multivariate, unstructured, multidimensional, tall, a large graph, set-typed, a dynamic graph, or an ML model.

Reusable 5-step strategy template

1. **Identify data type & task.** Name cardinality (tall? wide?), attribute types, and the analytic goal (overview, compare, find outliers, track over time).
2. **Preprocessing.** Name the *scalability challenge* explicitly (perceptual: overplotting; interactive: latency). Pick the matching tool: spatial/temporal **binning** + **aggregation** for perceptual scalability; a **data cube** / pre-aggregation for interactive scalability; **sampling** or **dimensionality reduction** where appropriate.
3. **Visual encoding.** Name the *marks* and *channels* and the *colour map* (sequential for ordered magnitude, diverging for signed, categorical for nominal).
4. **Interaction.** Name techniques: **linked (coordinated) views**, **brushing**, **filtering**, **details-on-demand**, zoom/pan, level-of-detail.
5. **Justify vs. alternatives.** Say why the chosen encoding beats a plausible competitor (e.g. binned map vs. raw points; choropleth vs. hexbin; line chart vs. heatmap).

Pitfall / point deduction

Deductions: not identifying the scalability challenge; proposing no linked views; ineffective processing (e.g. raw KDE on 10M points); doing only spatial *or* temporal binning when both are needed; not naming the visual encoding (marks/channels); not mentioning binning/aggregation; not naming the interaction design; non-optimal colour mapping (e.g. rainbow for magnitude).

Mini-example. 10M-row table of EU traffic accidents (ID, lon, lat, year): spatial + temporal binning (perceptual scalability) backed by a data cube (interactive scalability); two linked views, a hexbin/choropleth map (sequential colour, regions brushable) and a temporal line chart/histogram (year ranges brushable). Worked in full in Section 16.

13.4 Type 4 – Design & Validation Strategy (user study)

What it asks. Given a research topic, design a validation / user study: state hypotheses and identify independent, dependent, controlled, and confounding variables.

Answer strategy

1. **Hypothesis** (directional, falsifiable): “Encoding A yields faster/more accurate task performance than encoding B.”
2. **Independent variable (IV)**: what you manipulate (the encoding / technique), with its levels.
3. **Dependent variables (DV)**: what you measure – task completion *time*, *accuracy* / *error rate*, subjective preference, cognitive load, confidence.
4. **Controlled variables**: held constant (dataset size, screen, task type, training).
5. **Confounds**: learning/order effects (counterbalance, e.g. Latin square), fatigue, individual differences (within-subjects design).
6. State **design**: within- vs. between-subjects, #participants, analysis (ANOVA / t-test).

Pitfall / point deduction

Mixing up IV and DV. Forgetting to control dataset/task so the encoding is the only thing varying. No mention of order/learning effects or counterbalancing. Vague, non-measurable DVs.

Mini-example. “Are class-by-colour or class-by-shape scatterplots more effective?” IV = encoding (colour vs. shape vs. both); DV = class-membership judgment time & accuracy; control point count/overlap; counterbalance order.

13.5 Type 5 – Critique & Suggestions

What it asks. Discuss strengths/problems of a given visualization and suggest better encodings, using the design vocabulary.

Critique checklist

1. **Data-ink ratio** / chartjunk: is ink doing data work?
2. **Expressiveness**: does it show all and only the data (no spurious order on nominal data)?
3. **Effectiveness** (Stevens’ power law, channel ranking): position > length > angle/area > colour. Is the most important attribute on the strongest channel?
4. **Hidden transformations**: implicit aggregation, normalization, log axes, truncated baselines.
5. **Lie factor** / 3D distortion / non-zero baseline.
6. **Colour**: sequential vs. diverging vs. categorical; perceptual uniformity; rainbow problems.
7. **Channel interference** / ordering.
8. **Suggest** a concrete alternative encoding.

Pitfall / point deduction

Listing principles without applying them to *this* chart. Forgetting to propose a concrete fix. Calling all 3D bad without naming the actual problem (occlusion, perspective distortion, foreshortening).

Mini-example. A rainbow choropleth of unemployment: rainbow is not perceptually ordered nor uniform $\Rightarrow$ replace with a single-hue sequential map; mention simultaneous-contrast and colour-blind safety.

13.6 Type 6 – Alternatives (compare two views)

What it asks. Given two visualizations of the same data, discuss pros/cons (classic: parallel coordinates vs. SPLOM).

Answer strategy

Make a 2-column pro/con table on shared axes: **scalability** (#dims, #points), **task fit** (pairwise correlation vs. multivariate profiles / clusters), **clutter** / **overplotting**, **axis ordering effects**, **interaction** needs. Conclude with “use X when task is . . . , Y when . . .”.

Pitfall / point deduction

Only listing features of each without comparing them on the same criteria, or not tying the recommendation to a *task*.

Mini-example. SPLOM shows all pairwise correlations but costs $O(d^2)$ panels; PCC scales to more dimensions in one view and shows multivariate profiles/clusters but suffers ordering effects and line clutter.

13.7 Type 7 – Transfer / Sketch

What it asks. Transfer one representation to another and sketch it (e.g. a number array $\rightarrow$ histogram + boxplot).

Answer strategy

Extract the summary statistics the target needs, then draw axes and the marks. For a boxplot: sort, find min, Q1, median, Q3, max, fence outliers at 1.5 IQR. For a histogram: choose bins, count, draw bars. Always **label the axes and key values**.

Pitfall / point deduction

Unlabelled axes; wrong quartile placement; ignoring outliers; bins that hide the distribution shape.

Mini-example. Array {1, 2, 2, 3, 9}: median = 2, Q1 = 2, Q3 = 3, 9 is an outlier ($> Q3 + 1.5 \text{ IQR}$); histogram is right-skewed.

13.8 Type 8 – Mapping Data Items Between Views

What it asks. A data item highlighted in one view; mark the same item in a paired view. Pairs: scatterplot $\leftrightarrow$ PCC; PCC $\leftrightarrow$ star plot; heatmap $\leftrightarrow$ grouped/stacked bars; node-link $\leftrightarrow$ adjacency matrix; treemap $\leftrightarrow$ rectilinear node-link tree.

Answer strategy

Recover the item's *coordinates* / *attribute values* from the source view, then find the mark with those same values in the target. A scatter point (x, y) becomes the PCC polyline passing through x on axis 1 and y on axis 2; a heatmap cell becomes one bar in the corresponding bar group; a node becomes the matching row+column of the adjacency matrix; a treemap rectangle becomes the corresponding leaf in the node-link tree.

Pitfall / point deduction

Reading the wrong axis in PCC; confusing a node's row/column in the adjacency matrix; matching by colour alone when colour is reused.

Mini-example. Outlier point top-right of a scatter $\rightarrow$ the PCC polyline that is high on both corresponding axes.

13.9 Type 9 – List / Enumerate

What it asks. Enumerate concepts: raw-data issues; scalability factors; latency sources; graph-drawing aesthetics; no-preprocessing overplotting fixes; model-agnostic interpretability methods; set-typed visualization techniques; validation methods; typical user-study dependent variables.

Answer strategy

Give a *bulleted* list with a one-line gloss per item; aim for breadth (many items) over depth. The complete memorizable lists are in Section 15.

Pitfall / point deduction

Writing prose instead of a list; giving too few items; padding with synonyms of the same concept.

Mini-example. Overplotting fixes needing no preprocessing: transparency (alpha), smaller marks, jitter, open glyphs — vs. binning/DR which *do* require preprocessing.

14 Algorithm Cheat-Sheet

A compact one-row-per-algorithm reference for the “describe / sketch / MCQ” question type. Detailed treatments are in Sections 7–11; this is the revision sheet. Sketch boxes follow for the algorithms where a drawing matters.

Algorithm	Core idea	Key hyperparameters	Key properties / pitfalls
Recursive subdivision (mosaic)	Split a rectangle by category proportions, alternating axes per variable	Variable order; spacing	Tile area = joint frequency; order changes layout; reads marginals/conditionals
Recursive subdivision (treemap)	Recursively nest rectangles sized by a hierarchy’s leaf values	Tiling (slice-and-dice, squarified); layout order	Area = value; squarified gives good aspect ratios; deep hierarchies hard to read; area is a weak channel
Just-noticeable-difference (JND)	Smallest stimulus change a viewer can detect	(Weber fraction k)	Weber’s law: $\Delta I/I = k$; motivates perceptually uniform colour/size scales
Scagnostics	Graph-theoretic measures summarizing a scatterplot’s shape	(Binning grid; measure set)	Outlying, skewed, clumpy, sparse, striated, convex, skinny, stringy, monotonic; used to rank SPLOM panels
Class consistency	Quality metric: do class labels agree with spatial structure of a projection?	Distance/centroid criterion	Higher = classes form separable clusters; used to evaluate/rank DR layouts
PCA	Linear projection onto orthogonal directions of max variance	#components k	Linear, deterministic; preserves global variance; axes interpretable; misses nonlinear structure

Algorithm	Core idea	Key hyperparameters	Key properties / pitfalls
MDS	Embed so low-D distances match high-D (dis)similarities	metric vs. non-metric; #dims; stress	Preserves pairwise distances; classical MDS $\approx$ PCA; can be slow; stress measures distortion
t-SNE	Match neighbour probability distributions (high-D vs. low-D), minimize KL	perplexity ; learning rate; iterations; seed	Nonlinear, stochastic; preserves local clusters; <i>cluster sizes & between-cluster distances not meaningful</i>
UMAP	Build a fuzzy k -NN graph, optimize a low-D layout of it	$n_neighbors$; min_dist ; #dims; seed	Nonlinear, stochastic; faster than t-SNE; better global structure; sizes/distances still unreliable
Hexagon binning	Aggregate points into hexagonal bins, encode count by colour	bin size / #bins	Fixes overplotting (perceptual scalability); hexagons tile uniformly, less axis bias than squares
Kernel density estimation	Sum smooth kernels over points to estimate a density	bandwidth ; kernel shape	Bandwidth too small = spiky, too large = oversmoothed; $O(n)$ per eval – costly on tall data
Multi-dim tables / data cubes	Pre-aggregate measures over dimension combinations for fast queries	dimensions; hierarchies; measures	Interactive scalability: slice/dice/roll-up/drill-down in $O(1)$; storage grows with dimensions
Parallel aggregation	Distribute aggregation over partitions, combine partial results	#partitions	Scales aggregation to tall data; basis of data-cube construction; needs associative aggregates
Feature vectors (neuron activations)	Represent an input by a layer’s activation vector	layer choice; (DR for display)	Used to inspect what a network “sees”; high-D, usually projected with t-SNE/UMAP
Local perturbations (LIME)	Perturb one input, fit a simple local surrogate	#samples; kernel width; surrogate	Model-agnostic, <i>local</i> explanation; unstable to sampling; not globally faithful
Partial dependence (PDP)	Average model output while sweeping one feature over its range	feature(s); grid resolution	Global, model-agnostic; assumes feature independence; can hide interactions (ICE curves fix this)
Counterfactual explanation	Smallest input change that flips the prediction	distance metric; target class; sparsity	“Change X to get outcome Y”; should be close, sparse, actionable, plausible; many valid counterfactuals

Algorithm	Core idea	Key hyperparameters	Key properties / pitfalls
Force-directed layout	Springs on edges + repulsion on nodes, minimize energy	spring constant; repulsion; iterations; temperature	Good for small/medium graphs; stochastic; $O(n^2)$ per step (Barnes–Hut speeds it); can get stuck in local minima
Layered (Sugiyama) layout	Assign nodes to layers, order to cut crossings, assign coordinates	layer assignment; crossing-reduction passes	For DAGs/hierarchies; minimizes edge crossings; long edges via dummy nodes
Graph morphing	Smoothly interpolate between two layouts of a graph	interpolation; duration	Preserves the mental map during transitions; animate node positions
Foresighted layout	Lay out a dynamic graph anticipating future states for stability	time window; stability weight	Trades layout quality vs. stability across time steps; keeps the mental map over a sequence

PCA – 1D projection sketch

Centre the data, find the direction of maximum variance (first eigenvector of the covariance matrix), and project points onto it.

Sketch: Draw an elongated 2D point cloud. Draw an arrow through its long axis (PC1). Drop a perpendicular from each point onto that line; the feet of the perpendiculars are the 1D coordinates. PC2 is orthogonal and discarded.

Recursive subdivision – mosaic / treemap sketch

Both split a rectangle proportionally and recurse.

Sketch: Mosaic: split the full width by variable A's proportions, then split each resulting column's height by variable B's conditional proportions; tile area = joint frequency. Treemap: split the rectangle by the top-level children's summed values, then recurse inside each child; leaf area = value.

Hexagon binning sketch

Sketch: Overlay a hexagonal grid on the scatterplot, count points per hexagon, and fill each hexagon with a sequential colour by count. Replaces a dense black blob with a readable density field.

KDE – bandwidth sketch

Sketch: Place a small bump (kernel) on each data point on the x-axis and sum them into a smooth curve. Show three curves: tiny bandwidth (spiky, overfit), good bandwidth (smooth bimodal), huge bandwidth (one flat hump, oversmoothed).

Force-directed layout sketch

Sketch: Nodes as charged particles repel each other; edges act as springs pulling connected nodes together. Iterate until the energy settles. Draw a small graph with well-separated nodes and

short, roughly equal-length edges, clusters visibly grouped.

Layered (Sugiyama) layout sketch

Sketch: Draw horizontal layers (rows). Place each node in a layer by its topological rank; route all edges downward; reorder nodes within layers to minimize crossings; insert dummy nodes for edges spanning multiple layers.

Counterfactual explanation sketch

Sketch: Plot the decision boundary in feature space with the query point on the “rejected” side. The counterfactual is the nearest point across the boundary; draw the short arrow from query to that point – the minimal, sparse change that flips the outcome.

Partial dependence sketch

Sketch: x-axis = one feature’s value, y-axis = average model prediction. For each grid value of the feature, set it for all rows, predict, and average. Plot the resulting curve; its slope shows the feature’s marginal effect.

15 List-Question Quick Reference

Memorable lists for the “enumerate” question type. Each item has a brief gloss; in the exam, give the list and a one-line gloss per item.

15.1 Raw-data issues in data tables

- **Missing values** – empty cells / NA; need imputation or removal.
- **Errors / outliers** – typos, sensor faults, impossible values.
- **Duplicates** – repeated records inflate counts.
- **Inconsistent units / formats** – km vs. miles, date formats.
- **Inconsistent encoding** – “M”/“Male”/“1” for same category.
- **Wrong data types** – numbers stored as text, categorical as int.
- **Mixed granularity** – rows at different aggregation levels.
- **Non-tidy structure** – values in column names, multiple variables per column.
- **Skew / heavy tails / scale differences** – may need log or normalization.
- **Imbalanced classes** – distorts colour/area perception.

15.2 Factors affecting scalability

- **Number of data items** (rows) – overplotting, slow rendering.
- **Number of dimensions / attributes** – curse of dimensionality, too many axes/panels.
- **Number of categories** – exhausts distinguishable colours/shapes.
- **Display resolution / screen pixels** – pixel budget limits marks.
- **Human perception** – limited channels, pre-attentive capacity (**perceptual scalability**).
- **Rendering / computation budget** – frame rate, memory.
- **Interaction latency** – query/response time (**interactive scalability**).

15.3 Sources of latency

- **Data transfer / network** – moving data to the client.

- **Querying / aggregation** – on-the-fly computation over big data.
- **Rendering** – drawing many marks each frame.
- **Layout computation** – e.g. force-directed iterations, DR.
- **Disk / IO** – loading from storage.
- **Interaction round-trip** – client $\leftrightarrow$ server latency on brush/filter.

15.4 Aesthetic criteria for graph drawing

- **Minimize edge crossings.**
- **Uniform edge length** / minimize total edge length.
- **Even / symmetric node distribution**; display symmetries.
- **Angular resolution** – large angles between edges at a node.
- **Minimize edge bends**; few orthogonal bends.
- **Avoid node/edge overlap.**
- **Cluster separation** – communities visually grouped.
- **Consistent flow direction** (for directed/layered graphs).
- **Preserve the mental map** across changes (dynamic graphs).

15.5 Overplotting fixes that need no preprocessing

- **Transparency (alpha blending)** – density shows through.
- **Smaller marks / thinner lines.**
- **Open glyphs** (outlines instead of filled).
- **Jitter** – small random offset to separate coincident points.
- **Subsampling on the fly** (draw a random subset).

Contrast

Binning/hexbin, KDE, aggregation, and dimensionality reduction *do* need preprocessing – do not list them as “no-preprocessing” fixes.

15.6 Model-agnostic AI interpretability methods

- **Partial dependence plots (PDP)** – average marginal effect of a feature.
- **Individual conditional expectation (ICE)** – per-instance PDP curves.
- **Permutation feature importance** – accuracy drop when a feature is shuffled.
- **LIME** – local surrogate from input perturbations.
- **SHAP** – Shapley-value attribution of the prediction.
- **Counterfactual explanations** – minimal change to flip outcome.
- **Local surrogate / global surrogate models.**
- **Accumulated local effects (ALE)** – like PDP but robust to correlated features.

15.7 Techniques to visualize set-typed data

- **Venn / Euler diagrams** – overlapping regions; poor beyond 3–4 sets.
- **UpSet plots** – matrix of intersections with bar charts; scales to many sets.
- **Set membership matrix** – rows = elements, columns = sets.
- **Linear / radial set diagrams** (LineSets, Kelp diagrams, BubbleSets) – overlays on a spatial layout.
- **Aggregate bar charts** of set / intersection sizes.
- **Parallel sets** – flows between categorical set memberships.

15.8 Methods for validation of visual encoding and/or interaction design

- **Controlled experiment / user study** – measure time & accuracy.
- **Qualitative usability study / think-aloud.**
- **Expert review / heuristic evaluation.**
- **Case studies** with domain experts in the field.
- **Computational / algorithmic benchmarks** – e.g. quality metrics (stress, class consistency, scagnostics).
- **Crowdsourced experiments** (e.g. MTurk) for many participants.
- **Analytical evaluation** against perceptual theory (expressiveness/effectiveness).
- **Log analysis** of real usage; A/B tests.

15.9 Typical dependent variables in user studies

- **Task completion time** (speed).
- **Accuracy / error rate** (correctness).
- **Confidence** ratings.
- **Subjective preference / satisfaction.**
- **Cognitive load** (e.g. NASA-TLX).
- **Insights found / recall / memorability.**
- **Eye-tracking measures** (fixations, scan path).

16 Practice Questions with Model Answers

Worked practice questions spanning all nine question types. Cover the question, attempt your own answer, then check the model answer.

Practice Q1 – Literacy: confusion matrix (T/F)

A 4-class action-recognition confusion matrix (rows = true class, columns = predicted) has rows boxing, handwaving, jogging, running. Row sums are 100. Diagonals: boxing 98, handwaving 95, jogging 60, running 80. Cell (boxing, handwaving) = 0; cell (jogging, running) = 30. Mark each True/False: (1) Boxing is never confused with handwaving. (2) Jogging is the hardest class to classify. (3) Jogging is most often confused with running. (4) Running is classified perfectly.

Model answer. (1) **True** – cell is 0. (2) **True** – smallest diagonal (60/100) = lowest recall. (3) **True** – its largest off-diagonal cell (30) is the running column. (4) **False** – diagonal 80, so 20% misclassified. Always trace the exact cell; remember rows are the true classes.

Practice Q2 – Algorithms: counterfactual (T/F)

A loan model rejects an applicant. Mark which statements about *counterfactual explanations* are correct: (1) A good counterfactual changes as few features as possible. (2) There is exactly one counterfactual per instance. (3) Counterfactuals are model-agnostic and need no access to model internals. (4) The counterfactual should be plausible / actionable (e.g. not “reduce age by 10 years”).

Model answer. Correct: (1), (3), (4). (2) is **False** – many counterfactuals can exist (Rashomon effect); good methods report close, sparse, plausible ones. (3) holds because counterfactuals only query input→output behaviour.

Practice Q3 – Strategy: 10M traffic-accident records

A table has 10 million European traffic-accident records: (ID, longitude, latitude, year). Design an interactive web visualization to explore the distribution *spatially and temporally*. How would you preprocess?

Model answer (5-step template).

1. **Data type & task.** Tall (10M rows) spatio-temporal point data; task = explore spatial and temporal distribution and their interaction.
2. **Preprocessing / scalability.** The challenge is twofold: *perceptual scalability* (10M points overplot any map) and *interactive scalability* (raw queries are too slow). Solution: **spatial binning** (hexbin grid or aggregate to NUTS regions) and **temporal binning** (by year), with counts pre-aggregated into a **data cube** over (space-bin $\times$ year) for $O(1)$ slicing. Both spatial *and* temporal binning are required.
3. **Visual encoding.** Two views. Spatial: **choropleth** (regions) or **hexbin map**, marks = area cells, channel = colour by accident count using a **sequential** colour map. Temporal: **line chart** (or histogram), marks = line/bars, channel = position (x = year, y = count).
4. **Interaction. Linked (coordinated) views:** **brush** a year range on the temporal chart to update the map, and **brush** regions on the map to filter the temporal chart (cross-filtering); plus zoom/pan and **details-on-demand** on hover.
5. **Justify vs. alternatives.** Binned map beats raw scatter (no overplotting); data cube beats live aggregation (low latency); sequential beats rainbow (perceptually ordered). KDE on raw 10M points would be too slow / ineffective.

Deductions to avoid: not naming the scalability challenge; no linked views; only spatial or only temporal binning; missing marks/channels; rainbow colour map; no mention of aggregation/data cube.

Practice Q4 – Validation: user study for scatterplot class encodings

Design a user study to evaluate the effectiveness of visual encodings of classes/sets in *static* scatterplots.

Model answer. Hypothesis (H1): colour-coded classes yield faster and more accurate class-separation judgments than shape-coded classes; **H2:** effectiveness degrades as the number of classes grows. **IV:** class encoding (colour / shape / colour+shape) and #classes (3 / 6 / 9). **DV:** task completion time, accuracy/error rate, confidence, preference. **Controlled:** number of points, degree of overlap, point size, display, task wording, training. **Confounds:** colour-vision deficiency (screen participants), learning/order effects (counterbalance with a Latin square), fatigue. **Design:** within-subjects, e.g. 24 participants; tasks = “which class is denser here?” / “find the outlier of class X”; analyze with repeated-measures ANOVA. Report effect sizes.

Practice Q5 – Critique: rainbow choropleth

A regional-unemployment-rate map uses the rainbow (jet) colour scale. Critique it and suggest a fix.

Model answer. Problems: (i) **rainbow is not perceptually ordered** – viewers cannot rank hues, so high vs. low is ambiguous; (ii) **not perceptually uniform** – equal data steps produce unequal perceived steps (false boundaries at yellow/cyan); (iii) **not colour-blind safe**; (iv) mixes hue and lightness, causing **channel interference**; (v) rate (a magnitude) should use an ordered channel. Fix: a **single-hue (or ColorBrewer) sequential** map ordered by lightness; consider a **diverging** map if comparing against a national-average reference. Also watch simultaneous contrast between adjacent regions and the hidden choropleth bias toward large-area regions (consider rate per capita / a cartogram).

Practice Q6 – Alternatives: parallel coordinates vs. SPLOM

The same 8-dimensional dataset is shown as parallel coordinates (PCC) and as a scatterplot matrix (SPLOM). Compare them.

Model answer.

- **Dimension scalability:** PCC = one axis per dimension (handles many dims in one view); SPLOM = $O(d^2)$ panels (64 here), shrinking fast.
- **Task fit:** SPLOM shows *all pairwise* correlations directly; PCC shows *multivariate profiles, clusters, outliers* and correlation only between *adjacent* axes.
- **Clutter:** PCC suffers heavy line overplotting on tall data; SPLOM suffers point overplotting per panel.
- **Ordering:** PCC depends strongly on **axis order** (non-adjacent relations hidden); SPLOM is order-insensitive.
- **Interaction:** PCC benefits from brushing + axis reordering; SPLOM from linked brushing across panels.

Recommendation: SPLOM for pairwise-correlation tasks at moderate d ; PCC for profile/cluster/outlier exploration at higher d (with reorderable axes).

Practice Q7 – Transfer/sketch: array $\rightarrow$ histogram + boxplot

Given the array $\{4, 5, 5, 6, 6, 6, 7, 8, 20\}$, sketch a histogram and a boxplot.

Model answer. Sorted $n = 9$: median = 6 (5th value); lower half $\{4, 5, 5, 6\} \Rightarrow Q1 = 5$; upper half $\{7, 8, 20\} \Rightarrow Q3 = 8$; IQR = 3; upper fence = $8 + 1.5 \cdot 3 = 12.5 \Rightarrow 20$ is an **outlier**; whisker max (non-outlier) = 8.

Sketch: Boxplot: box from 5 to 8, median line at 6, lower whisker to 4, upper whisker to 8, a separate dot at 20 marked as outlier. Histogram (bins of width 2): tall bar around 5–6, smaller bars at 4 and 7–8, an isolated low bar near 20 – right-skewed.

Practice Q8 – Mapping: scatterplot point $\rightarrow$ PCC polyline

A scatterplot of (weight, mpg) highlights one outlier car: high weight, low mpg. Mark the same car in a parallel-coordinates plot whose axes are, left to right, mpg, weight, horsepower.

Model answer. Recover its values: low on mpg, high on weight (horsepower unknown from the scatter, but heavy cars are typically high). The car's **polyline** is the one that is **low on the mpg axis and high on the weight axis** (and, for a heavy car, high on horsepower). Trace the single line crossing those positions and highlight it.

Sketch: Three vertical axes; one bold polyline starting low on mpg, rising to the top of the weight axis, staying high on horsepower; other lines faded.

Practice Q9 – MCQ: t-SNE properties

Which statements about t-SNE are correct? (a) It is a linear projection. (b) Perplexity balances attention to local vs. global structure. (c) Distances between well-separated clusters are quantitatively meaningful. (d) Different random seeds can give different layouts.

Model answer. Correct: **(b), (d)**. (a) is false (nonlinear); (c) is false (between-cluster distances and cluster sizes are *not* reliable). t-SNE is stochastic, so set a seed for reproducibility.

Practice Q10 – MCQ: PCA vs. UMAP

Pick the correct pairings. (a) PCA axes are interpretable directions of maximum variance. (b) UMAP is deterministic. (c) UMAP's `n_neighbors` trades local detail for global structure.

(d) PCA can capture nonlinear manifolds.

Model answer. Correct: **(a), (c)**. (b) false – UMAP is stochastic. (d) false – PCA is linear; use t-SNE/UMAP/MDS or kernel PCA for nonlinear structure.

Practice Q11 – MCQ: force-directed layout

Which hold for force-directed graph layout? (a) It is deterministic regardless of initialization. (b) It models edges as springs and nodes as mutually repelling charges. (c) Naive computation is $O(n^2)$ per iteration; Barnes–Hut approximates it. (d) It guarantees the global minimum-energy layout.

Model answer. Correct: **(b), (c)**. (a) false – the result depends on the random initial placement. (d) false – it can settle in a local minimum.

Practice Q12 – List: overplotting fixes with no preprocessing

List simple solutions to overplotting that require *no* data preprocessing.

Model answer. Transparency (alpha), smaller marks, open glyphs, jitter, on-the-fly subsampling. (Hexbin, KDE, aggregation, and DR are effective but *do* require preprocessing, so they do not count.)

Practice Q13 – Algorithms: sketch 1D PCA

Sketch the 1D PCA projection of an elongated 2D cloud tilted at about 30° .

Model answer.

Sketch: Draw the tilted elliptical cloud. Draw PC1 as an arrow along its long axis (max variance). Project each point perpendicularly onto PC1; the feet of those perpendiculars are the 1D coordinates. PC2 (orthogonal, short axis) is discarded.

PCA is linear and deterministic; PC1 captures the most variance.

Practice Q14 – List: model-agnostic interpretability methods

Name model-agnostic methods to interpret an ML model.

Model answer. PDP (average marginal effect), **ICE** (per-instance curves), **permutation feature importance**, **LIME** (local surrogate), **SHAP** (Shapley attribution), **counterfactual explanations**, **ALE plots**, and global **surrogate models**. All treat the model as a black box, querying only inputs→outputs.